

SWAT-a-Palooza



What we accomplished in just 2 days



36

Ideas



20

Completed Projects



35+

Participants across Dev, QE, UX
and Product

All contributing to building **Solutions OS**

JARVIS-A11Y-FIXER

Jarvis + Kiro + Playwright = Fixes in minutes, not days



Soma Sundaram
Lead Product Engineer



Harish
Senior Quality Engineer



The Problem

50+ A11Y tickets blocking compliance, each taking 30 min to 60 min to fix manually.



What We Built

An agent that deploys verified accessibility fixes in **3 minutes** with human approval.



The Impact

13x faster, zero failures, lowered skill barrier, clears the backlog in an afternoon.



The Bigger Picture

A reusable pattern for any "ticket code change deploy verify" workflow.

The Problem

Challenges in Manual Accessibility Remediation



Growing Backlog

50+ open A11y tickets in the Solutions backlog (e.g., GAM-15093), creating a significant technical debt.



Manual Complexity

A 30-60 min manual process per fix: reading, finding, understanding SAIL, changing, deploying, and verifying.

Buried in Code

Most fixes are simple parameter additions or removals, but they are often **buried in complex SAIL**, making them hard to locate and modify manually.

GSS 1.3.1 A11y Findings					
+ @					
> Key details					
Child work items					
58% Done					
Work	Prior...	Story ...	Assi...	Status	
GAMS-540 Inappropriate use of hidden labels in AS_VM_FM_profileDetailsVendorDetails interface	☐ N...	2	Uni	BACKLOG	
GAMS-539 Icon has caption and duplicate text alternative	☐ N...	None	Uni	BACKLOG	
GAMS-534 [SETTINGS] Button controls have duplicated names	☐ N...	None	Uni	BACKLOG	
GAMS-533 [SETTINGS] Linked card uses unnecessary accessibility text	☐ N...	None	Uni	BACKLOG	
GAMS-531 Image of text is used	☐ N...	None	Uni	BACKLOG	

The Solution



A Kiro power that reads a Jira ticket and deploys the fix end-to-end - "Fix GAMS-533" → Done in 3 minutes

Architecture Workflow



How It Works (8 Phases)

PHASE 1

Ticket Intake

Read Jira ticket, classify fix pattern

PHASE 2

KB Mapping

Locate interface via Jarvis KB

PHASE 3

Path Planning

Plan navigation path to live UI

PHASE 4

Pre-Fix Audit

Verify issue exists in accessibility tree

PHASE 5

SAIL Discovery

Get SAIL, locate target component

PHASE 6

Apply & Approve

Apply fix, show before/after, get human approval

PHASE 7

Deployment

Deploy via Jarvis (inspection + deployment)

PHASE 8

Post-Fix Confirm

Re-open browser, confirm fix landed

Governance & Limitations

Human approves before **every** deployment. Agent refuses **Tier 3** (structural) fixes.

For you

Recent

Starred

Apps

Plans

Spaces

Recent

Government Acquisition M...

Recent

GAM Source Selection and Vend...

GAM Requirement and Award Ma...

GAM Requirement Management

GAMS Clause Automation

GAM Suite

View all boards

Government Acquisition Managem...

Internal Assistance

Engineering Assistance

Stratus Platform

More spaces

Filters

Dashboards

Teams

Goals

Projects

Customise sidebar

Spaces / Government Acquisiti... / GAM-15093 / GAMS-533

Key details

General Bug Fields Verification & Validation

Description

SETUP

The card uses accessibility text that duplicates the text used in the card and instructs users to "press enter to navigate to" + the text used in the card. The destination or action of the link is already present in the card text and screen reader users already know they need to press the Enter key to activate the link because it is defined as a link. The information conveyed to screen reader users is redundant and can be perceived as being condescending.

WCAG Requirement: 1.3.2 (A) Meaningful Sequence

Settings Interfaces:

Card links in the left rail navigation

Card links used on the Welcome screen

STEPS

Start the screen reader and navigate to the listed cards using the TAB or SHIFT+TAB keys

OBSERVATION

The screen reader announces the text present in the card and announces "press enter to navigate to" + the text used in the card and identifies the element as a link.

EXPECTATION

The screen reader announces only the visible text in the card and announces the element role of "link".

REMEDIATION

Remove the accessibility text parameter value from the alcardLayout.

Acceptance Criteria

Add text

Subtasks

Add subtask

Linked work items

Add linked work item

Create

Ask Rovo

2

Backlog

Improve Bug

Details

Assignee

Unassigned

Assign to me

Pull Request

Add URL

Parent

GAM-15093 GSS 1.3.1 Atty Findin

Team

GAM Source Selection and Vendor Management Team

Group

GOVERNMENT PROCUREMENT ...

SBU

Solutions

Product Owner

celine.guan

Reporter

Kurt Bunge

Priority

Not Set

Sprint

GSS Atty Bugs

Labels

ATTY-L

Components

GSS Settings: Accessibility

Development

More fields

Story Points

None

Affects versions

None

What Makes This Different

Full-Loop Deployment Automation

Moving beyond just "AI writes code" to a complete, autonomous cycle.



1. Reads Ticket

Analyzes Jira MCP to understand the reported issue.



2. Scans Codebase

Jarvis KB provides instant context—no 20-call exploration.



3. Verifies Issue

Uses Playwright accessibility tree to confirm the bug exists.



4. Deploys Fix

Automatically pushes the solution via Jarvis deployment API.



5. Confirms Success

Playwright post-fix snapshot validates the resolution.



Steering File Orchestration

Zero custom code. Fully portable as a Kiro Power.

SO WHAT

10x Efficiency Gain

From 30-60 min manual fixes to 3-5 min agent-deployed verified fixes



Safety Built-In

Only modifies <definition>, never business logic. Human gate before deployment.



Verification

Playwright confirms the accessibility tree before and after every fix.



Universal Scaling

Works across GSS, VM, GCW, and RM using Jarvis configuration.

50+ ticket backlog → clearable in a single afternoon

Covering 42 common patterns (parameter add/change/remove)

SO WHAT

Metric

Time per Tier 1 fix

50-ticket backlog

Developer skill required

Verification

Risk of breaking something

Manual

~40 min

33 dev-hours

Appian Dev
(knows SAIL, Appian functions & object)

Manual browser testing

Human error possible

With A11Y Fixer

~3 min

~2.5 hours

Any team member
(agent does the work)

Automated
(Playwright accessibility tree)

Human Intervention

SO WHAT

EPIC GAM-15093 (GSS 1.3.1 A11y Findings)

7 tickets triaged in one session:

Ticket	Result	Time
GAMS-533	✓ Fixed & verified	~3 min
GAMS-516	✓ Fixed (3 objects)	~5 min
GAMS-539	✓ Fixed (overlap)	—
GAMS-534	✖ Already fixed	~2 min (verified)
GAMS-540	✖ Already fixed	~2 min (verified)
GAMS-523	⚠ Flagged for manual	~3 min (investigated)
GAMS-531	⚠ Tier 3 refused	~1 min (located code)

4 deployments, 0 failures, all verified via Playwright accessibility tree

Atlas UX Designer Power Enhancements

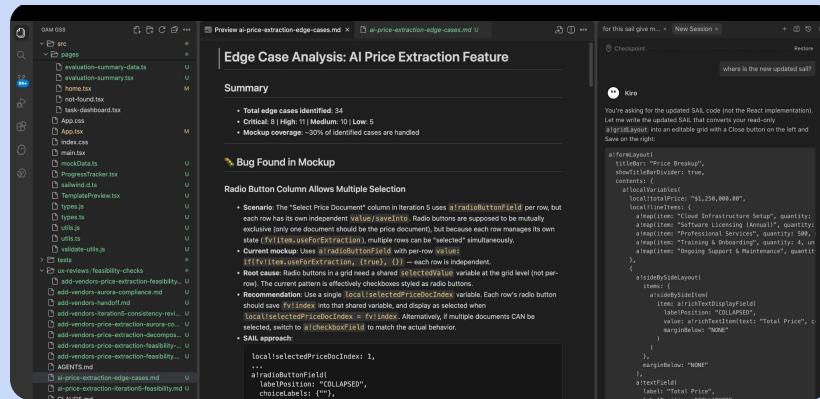


Vedant
User Experience
Designer

6 New Actions with standard template inputs and outputs !!!!!

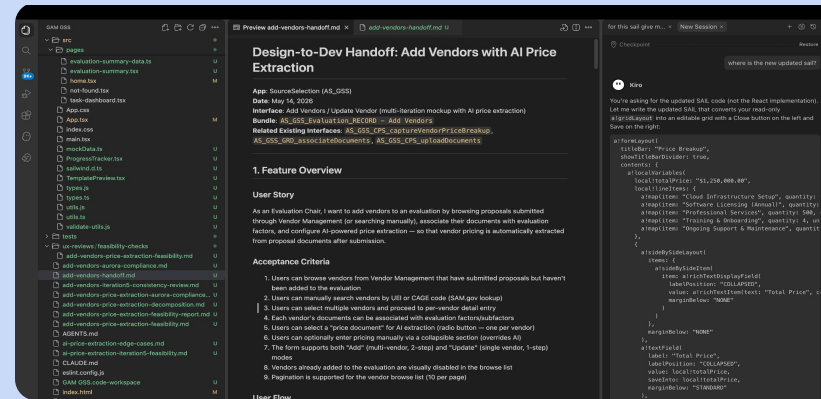
4. Edge Case Analysis

Systematically identifies missing elements: unhandled empty states, error conditions, loading states, and boundary cases. Generates "dev questions" for designer clarification. Output is a structured markdown report.



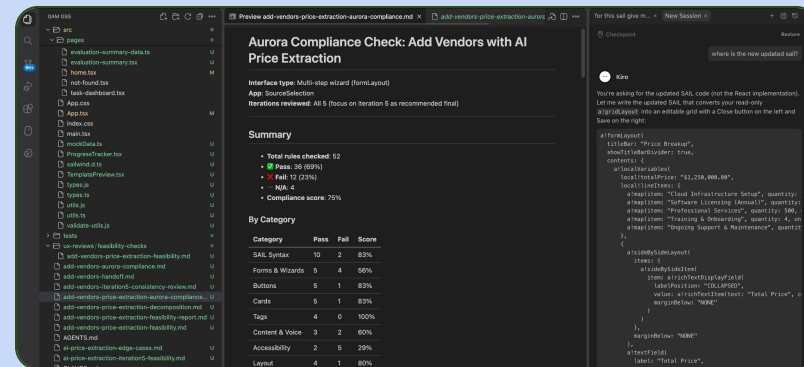
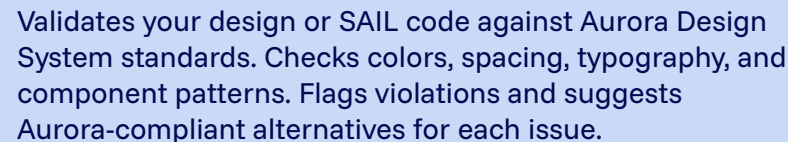
8. Design-to-Dev Handoff

Generates a complete implementation brief including component breakdown, data models, interaction specs, and validation rules. Translates UX designs into developer-ready spec documents.



6 New Actions with standard template inputs and outputs !!!!!

You describe a design or interaction pattern and it tells you what's achievable in SAIL vs. what's impossible or requires workarounds. It checks against known SAIL component limitations and platform behaviors. Useful for a reality check before handoff.



Atlas UX Designer Power Enhancements



Vedant
User Experience
Designer

6 New Actions with standard template inputs and outputs !!!!!

6. Design Consistency Review

Compares your new design against existing interfaces in the same Appian application. It checks for patterns, component choices, spacing, and labeling conventions. Flags inconsistencies to ensure a cohesive user experience.

The screenshot shows the 'Design Consistency Review: Add Vendors (Iteration 5 - AI Price Extraction)' interface. It compares the current design against existing interfaces in the application. The 'Summary' section shows 12 consistent patterns, 4 minor deviations, 5 inconsistencies, and 3 new patterns. The 'Layout Structure' table compares the current design against existing interfaces.

App Convention	New Interface	Status
Top-level layout	aiFormLayout_v2 with aiHeaderTemplate1ap6	aiFormLayout_v2v1
Title	"Add Vendors"	"Add Vendors"
Step	ifIn bundle keys (ifIn_AiVendorSetup, ifIn_AiVendorSetup2)	Hardcoded English strings (Step 1 of 2 - Browse vendors to add)

7. Component Decomposition Plan

Breaks complex interfaces into recommended structure of reusable SAIL expression rules. Outputs a tree structure with rule names, responsibilities, and data flow parameters.

The screenshot shows the 'Component Decomposition Plan: Add Vendors with AI Price Extraction' interface. It breaks down the current design into a tree structure of reusable SAIL expression rules. The 'Naming Convention' section lists rules like 'aiFormLayout_v2' and 'aiHeaderTemplate1ap6'. The 'Structural conventions' section lists rules like 'aiFormLayout_v2' and 'aiHeaderTemplate1ap6'. The 'Decomposition Tree' section shows the tree structure of the rules.

```
graph TD
    Root[Component Decomposition Plan: Add Vendors with AI Price Extraction] --> aiFormLayout_v2[aiFormLayout_v2]
    Root --> aiHeaderTemplate1ap6[aiHeaderTemplate1ap6]
    aiFormLayout_v2 --> aiFormLayout_v2v1[aiFormLayout_v2v1]
    aiHeaderTemplate1ap6 --> aiHeaderTemplate1ap6v1[aiHeaderTemplate1ap6v1]
```

TEA - Test Execution Agent



Team



Annamalai



Anupama



Asmita



Devin



Divya



Harish



Hanna



Ram



Rajesh



Suganya



Susheela

Test Execution Agent

AI-powered test execution for Appian Solutions

The Current State

- QEs spend hours manually verifying each story tickets against live environments
- Knowledge lives in people's heads — not reusable by automation
- Repetitive: login → navigate → click → screenshot → report → repeat

✦ Our Solution - TEA

Give Test Execution Agent a Jira Story ticket



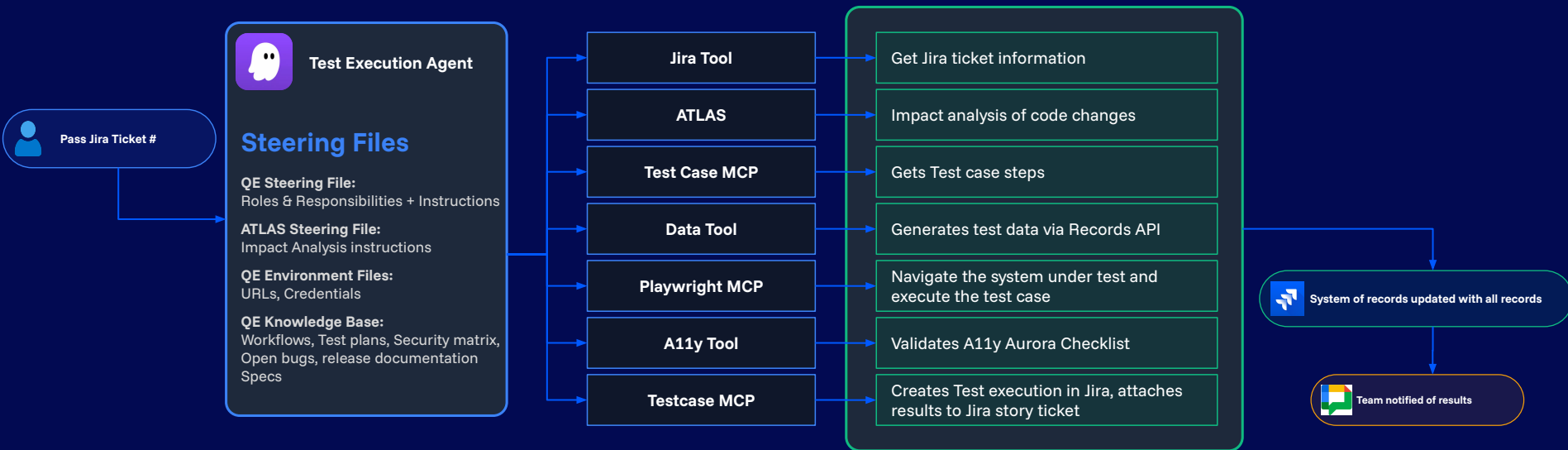
It verifies the ticket, executes the test case



Posts results with evidence back in Jira as comments, test execution run & Gchat

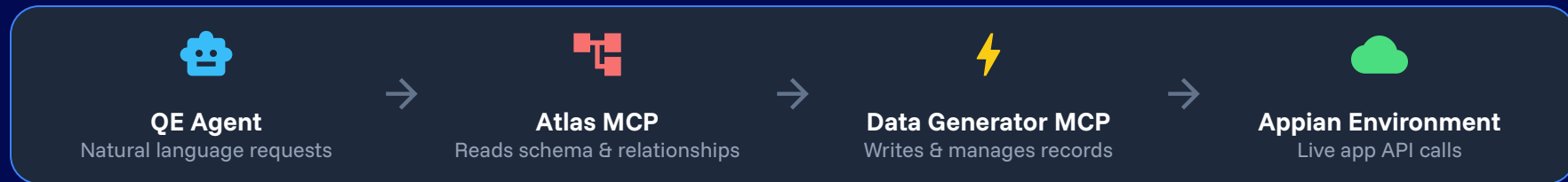
Built in 2 days by 11 people across US + India

System Architecture Flow



Atlas SQL Forge

Intelligent test data generation integrated into the QE Agent



What Makes It Intelligent

Schema from Atlas KB

Queries structures, FKs, and field mappings from application knowledge base.

Learns from Existing Data

Studies real data footprints to generate new matching patterns.

Understands Relationships

Resolves FK chains and creates complex hierarchies atomically.

Live Reference Data

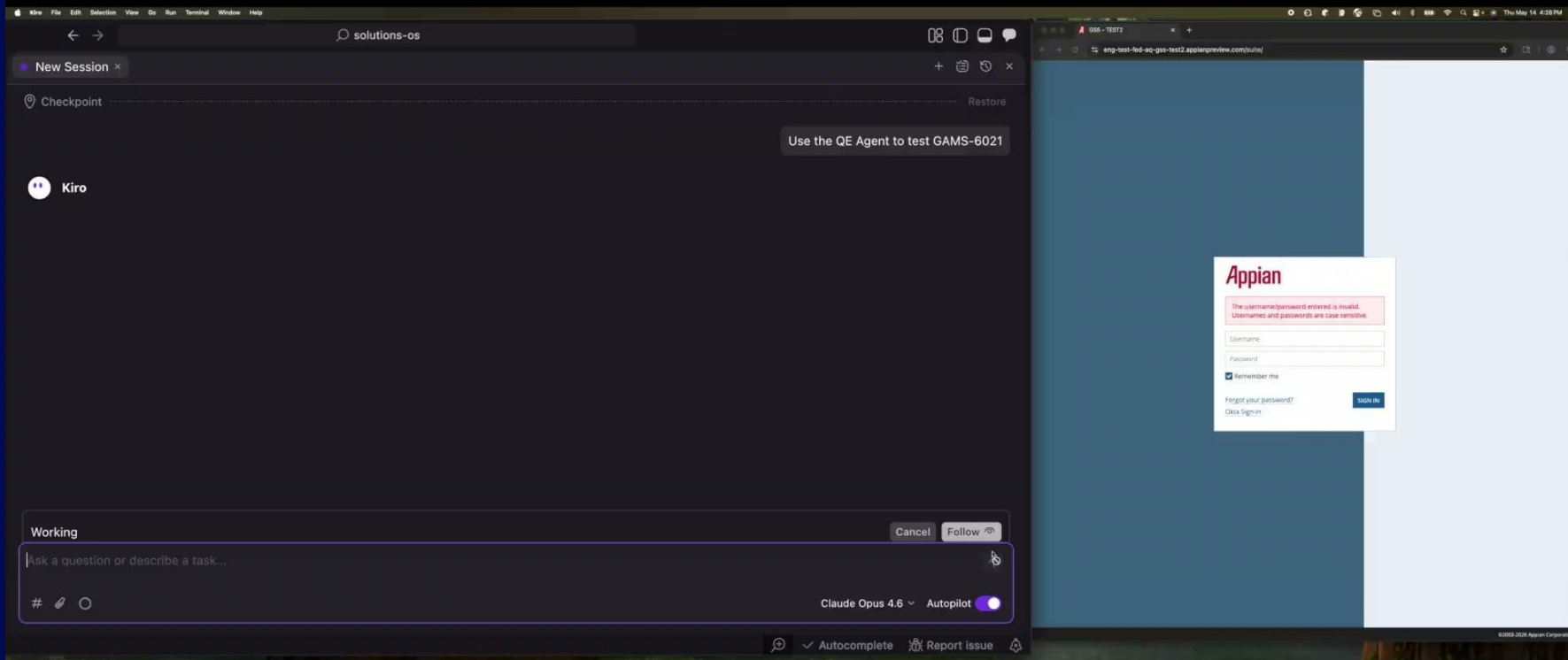
Queries status codes and lookups directly from the environment.

Workflow-Aware

Reads process models to understand data needs at each application state.

Self-Validating

Ensures ≥80% field coverage with documented reasoning.



Future (Next Steps)

Zero-Touch Verification using GitLab Pipelines

Workflow

- QE/Dev adds labels to tickets that can be picked by Agent for verification
- GitLab pipelines scan for tickets with label specified + status Verification & Validation
- Test Execution agent is invoked
- Results published & team notified

Fine Tuning

- Refine agent context to tune the agent for different Solutions, test scenarios, etc.

Atlas SQL Forge

Intelligent data tooling for Appian applications

Test Data

Bulk Data Generation

ERDs

SAIL-to-SQL



Ram



Kiro



Dineshkumar

The Problem

Appian applications have **50–100 interconnected database tables**. Today, creating test data is manual, error-prone, and takes hours.

A single "In Progress" record in Source Selection can require correct data in **7+ tables**. Today this is manual, error-prone, and takes hours.



The Solution

SQL Forge

SQL Forge understands the application's data model to automate complex workflows and ensure data integrity.



What SQL Forge Does



Test Data Setup

Creates records with all required child records, valid FKs, and proper field values.

- Regression testing - set up scenarios instantly
- Targeted functional testing - exact status
- Demo data - realistic, presentation-ready



Bulk Data Generation

Generates thousands of records as executable SQL scripts.

- Bulk data in required status and configuration
- Performance testing - scale distributions
- Respects FK constraints, batched INSERTs



ERD Generation

Creates Entity Relationship Diagrams automatically from live schema.

- Generates ERD for any app in seconds
- Automates ticket - always up to date
- Cross-application and Scoped views



SAIL to SQL

Converts performance-impacting SAIL to SQL Stored Procedures.

- Converts functionality without logic change
- Quick POC for verification
- Multiple DB support - MariaDB, Oracle

How It Works

Three systems working together — Atlas KB for intelligence, Data Generator MCP for execution, Appian for data



Atlas Knowledge Base

Parser extracts complete schema information from Appian packages.

*tables.json, relationships.json,
record_type_map.json, field_map.json*



Atlas MCP Server

Serves schema data via MCP protocol. Provides tools to query structures.

*get_record_type_map, get_field_map,
get_app_schema, get_insertion_order*



Data Gen MCP Server

Connects to live Appian via API. Handles CRUD, field validation, and rollbacks.

Supports nested records & session management.

Atlas KB
Schema + Maps



Atlas MCP
Schema Tools



SQL Forge
Agent Logic



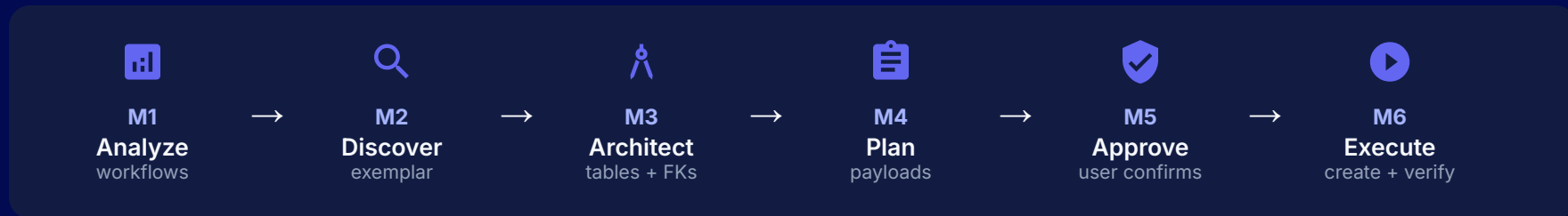
Data Gen MCP
Execution



Appian Env
Live Database

Intelligent Workflow

Every action follows a strict process — analyze, discover, plan, approve, execute



Learns from Atlas KB

Reads process models and business rules to understand what data should exist at each application state.



Uses Real Data as Template

Queries an existing record in the target status and studies its complete data footprint - then generates matching data.



Self-Validates

Checks field coverage ($\geq 80\%$), verifies FK integrity, documents reasoning for every value, gets user approval before executing.

The Impact

10x

Faster Conversion

Several days of manual SQL → hours

5x

Better Performance

SAIL → SQL
(GCA Clause Text Processing)

10x

Faster Data Load

Works with any app - zero config

5x

Faster Test Data Setup

Works with any app

✗ Before

- Manual INSERTs for 7+ tables & hours of debugging
- Guessing FK IDs & copying outdated scripts
- SAIL → SQL migration takes days

✓ After

- Live ref data & FK constraints handled automatically
- Learns from real data patterns; ERD in seconds
- SAIL to SQL in minutes

EXPLORER

OPEN EDITORS

Power: Atlas SQL Forge

DATA-GENERATOR-WORKBENCH

data-requests

2026-05-13_bulk-100-evaluations-perf-t...

2026-05-13_evaluation-setup-2vendors-...

2026-05-14_pta-evaluation-5vendors-r...

Atlas SQL Forge

Generate realistic, workflow-aware test data in Applan application environments. Creates records with va... by

Try powerUninstallCheck for updates

Description

#atlas#data-generator#test-data#applan

Generate realistic, workflow-aware test data in Applan application environments. Creates records with valid reference data, correct FK relationships, and proper insertion order.

MCP Configuration

Open powers config

Third-party powers disclaimer

Kiro powers provided by third parties operate independently from the core Kiro service and may be subject to their own license terms. Kiro does not test or evaluate these tools across all use cases, so users should install only from trusted sources and review any associated documentation and licensing.

data-generator-workbench

New Session

Generate ERD diagrams of your schema

Query and validate existing data

Rollback (soft-delete) any records created during a session

Onboarding — Let's follow the steps

The documentation specifies a 3-step onboarding process. Let me run through it now.

Step 1: Verify Atlas KB access

> Used Kiro power atlas-sql-forge list_applications

Atlas KB is accessible. I can see 15 Applan applications in the knowledge base. Now let's continue.

Step 2: Verify environment access

> Used Kiro power atlas-sql-forge list_users

Environment is reachable — 17 users available. Now for the final onboarding step.

Step 3: Load schema context

Let me pick one application to demonstrate. Based on the existing data-requests folder in your workspace (which references evaluations), SourceSelection looks like the most relevant app. Let me load its schema summary:

> Used Kiro power atlas-sql-forge get_schema_summary

Onboarding complete. Here's the full picture:

Onboarding Summary

Step	Status	Result
Atlas KB access	✓	15 applications available
Environment access	✓	17 users reachable
Schema context	✓	SourceSelection: 105 tables, 940 columns, 173 FKs, 302 reference data rows

Quick Example

Let me show you a simple schema exploration to demonstrate the power in action. I'll look up what valid evaluation statuses exist in the SourceSelection app:

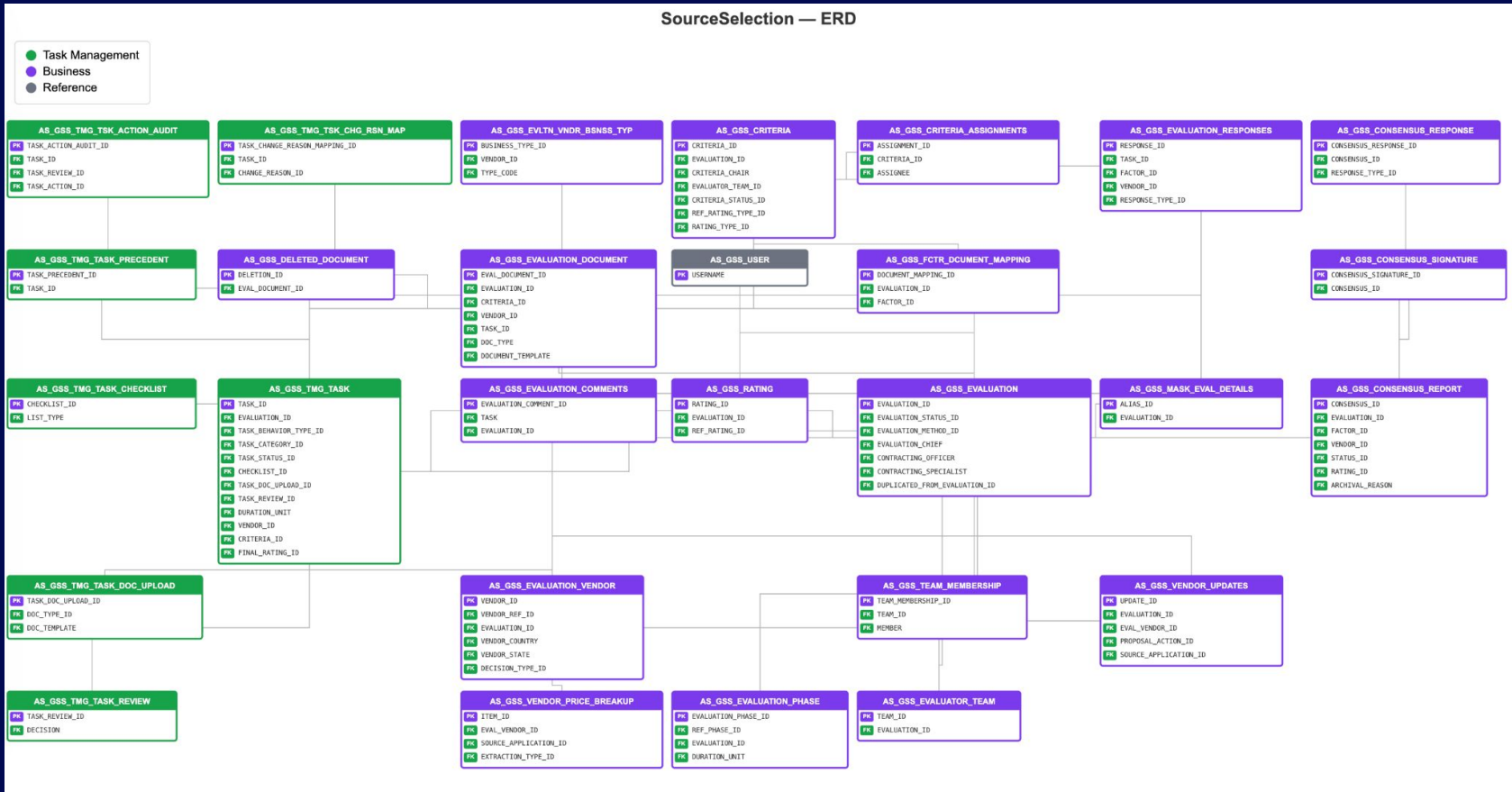
> Used Kiro power atlas-sql-forge get_reference_data

Create me an evaluation in setup status with two vendors and three criterias. Fill in all details for the vendors and cri

#Autopilot

AutocompleteReport issue

Sample ERD



A11y Audit Power

Automated Accessibility Auditing for Appian SAIL



Ganesh Raju

Senior Product Engineer

Why We Built A11y Power



Audits take 2-4 hours per interface. 500+ interfaces = 2000+ hours.



2 minutes per interface. Fully automated.



Knowledge scattered across Aurora website, GChat threads, and Jira across 8 projects.



All 3 sources unified into one automated check.



Same bugs keep coming back. No cross-app visibility.



Cross-app pattern database catches repeats automatically.



Inconsistent audits. Human fatigue misses rules. 95+ rules to remember.



Same 95+ rules applied every time, consistently.



Existing Jira a11y bugs go stale. No way to verify if they're still valid.



Validates existing Jira bugs against live code automatically.



Reports take 30+ minutes to write and format.



Auto-generated Google Doc in one step.

Quick Demo

mcp.json x

anesh.raju > .kro > settings > {} mcp.json > ...

User Config Workspace Config

173 }

TERMINAL

77 KiB | 584.Receiving ob
jects: 100% (7317/7317),
707.77 KiB | 584.Receivin
g objects: 100% (7317/731
7), 1001.10 KiB | 819.00
KiB/s, done.
Resolving deltas: 100% (5
407/5407), done.
bash-5.3\$ cd solutions-pi
pelines
bash-5.3\$ git checkout -b
a1ly-full-audit
Switched to a new branch
'a1ly-full-audit'
bash-5.3\$

power-appian-a1ly-audit

New Session x New Session x

Search in conversation (⌘F)

#a1ly-menu.md

Kiro

Hey! 🌟 I'm your **Appian A1ly Audit Assistant**. Here's what I can do:

What would you like to do?

#	Option	What It Does
1	Single Interface Audit	Deep audit of one interface + all its child interfaces (recursive). Includes Jira cross-reference.
2	Mockup Screenshot Audit	Drag a screenshot into chat — I'll check it for visual a1ly issues and map to SAIL rules.
3	Batch Audit — by Component	Find all interfaces using a specific component (e.g., all grids) and audit them.
4	Batch Audit — by Pattern	Audit all interfaces matching a name pattern (e.g., all FM_ forms).
5	Full Application Audit	Complete app-wide audit using tiered strategy (component scan + batched deep audit + pattern sampling + Jira cross-ref).
6	Paste SAIL Check	Paste SAIL code directly into chat — I'll check it against 50+ Aurora rules.
7	Component Search	Find which interfaces use a specific SAIL component. No audit, just discovery.
8	Audit from ZIP Export	Load a pre-exported ZIP file from your machine — no live Appian connection needed.
9	Package Audit	Load a package export ZIP and audit only the objects in that package.

Ask a question or describe a task...

Claude Opus 4.6 v Autopilot

Impact & Next Steps

Impact Today

Full app audit: 43 interfaces audited in **5 minutes** (vs 80+ hours manually)

Jira bug validation: Auto-verifies if existing bugs are still valid or already fixed against live code

Pattern Detection: Catches recurring accessibility violations across all projects

Report Generation: Auto-generates comprehensive Google Doc reports in one step

Next Steps

Daily A11y Pipeline Integration: Conduct daily, automated accessibility audit scans on the full application's latest package build.

Periodic Re-validation: Auto-Appian product changes

Rule Expansion: Add new audit rules as WCAG guidelines evolve

Team-wide Adoption: Standard QE tool across all Solutions teams

Jarvis Sweep

AI-Powered Code Cleanup for Appian SAIL
Solutions AI Hackathon — May 2026



The Problem

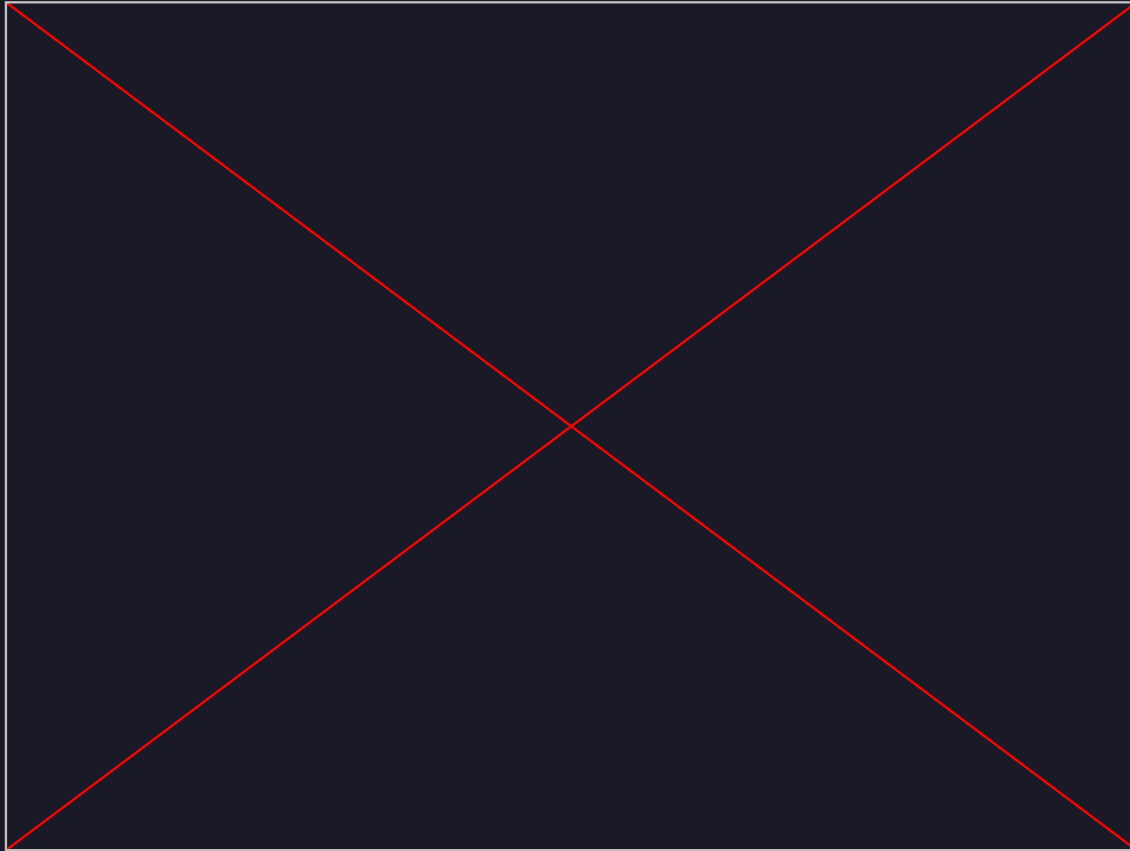
Keeping SAIL tidy can be tedious and time consuming, taking away from more important feature development

The need for

- Comments and documentation
- Consistent naming conventions
- Extracted variables for readability
- Valuable expression tests
- Keeping track of used variables
- Adding translation strings
- Configuring branding
- Finding and including constants



Introducing Jarvis Sweep



Descriptions

Interface Properties

Name *

RD_FM_VendorEvaluation

Description

Vendor evaluation form that displays vendor details (name, category, tier, status, score) in a read-only section and collects a new evaluation with rating (1-4), comments, and date. Uses RD.PL.calculateVendorScore for all display data. Uses CO.component.AF.CO.INP.textField for

Save In *

RD Rules & Constants

☐ Make Available Offline. [Learn how to design interfaces for offline use](#)

☐ Include in the design library. [Learn more ?](#)

UUID

_a-0000efbe-e882-8000-9e98-011c48011c48_1677748

CANCEL **OK**

Constants

IN Interface Definition



```
1  a!formLayout(  
2    contents: {  
3      a!sectionLayout(  
4        contents: {}  
5      ),  
6      a!sectionLayout(  
7        label: Section,  
8        contents: {  
9          a!textField(  
10             label: Name,  
11             value: ri!name,  
12             saveInto: ri!name,  
13             characterLimit: cons!RS_NS_INT_MAX_CHARACTER_LIMIT  
14           ),  
15          a!paragraphField(  
16             label: Description,  
17             value: ri!description,  
18             saveInto: ri!description,  
19             characterLimit: cons!RS_DESCRIPTION_CHAR_LIMIT  
20           )  
21        }  
22      )  
23    },  
24    buttons: a!buttonLayout(  
25      primaryButtons: {  
26        a!buttonWidget(  
27          label: Submit,  
28          submit: true,  
29          loadingIndicator: true,  
30        )  
31      }  
32    )  
33  )  
34  )
```

Form

Enter your details

Section

Name

0/255

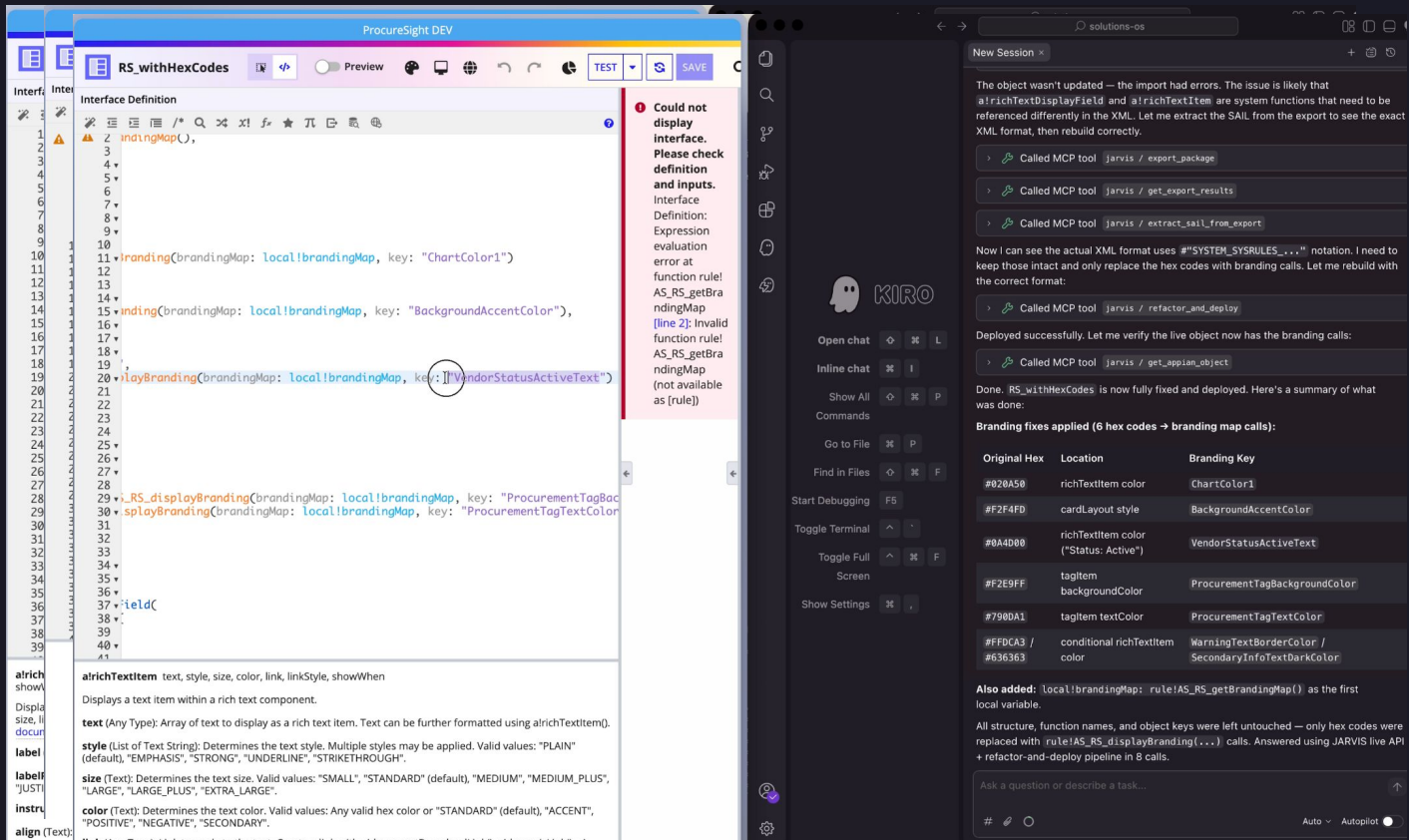
Description

0/1000

CANCEL

SUBMIT

Branding



How It Works

1. Export — Pull the object from Appian
2. Analyze — AI reads and understands the SAIL code
3. Clean — Jarvis rewrites package contents based on best practices
4. Deploy — Rebuild, inspect, and redeploy in one command

Under the hood:

- Automatic version UUID bumping on objects
- Package exports, unzip, rezip, and imports
- XML updates
- Inspection validation before deploy (aborts on errors)
- Polling until deployment completes
- Cleanup of intermediate files
- Summary of changes



Impact & What's Next

Impact:

- Reduces manual code hygiene time from hours to seconds
- Consistent code quality across all solution objects
- Safe deployments with built-in inspection gates

Supported today:

- Refactoring Expression Rules, Interfaces, & Branding
- Making Constants & Translation Strings

What's next:

- Operationalize Jarvis Sweep by merging into solutions-os and setting up docker build project
- Improve steering
- Add more functionality and increase reliability

Jarvis Sweep: Assert



AI-Powered Expression Rule Tests for Appian SAIL

Solutions AI Hackathon — May 2026

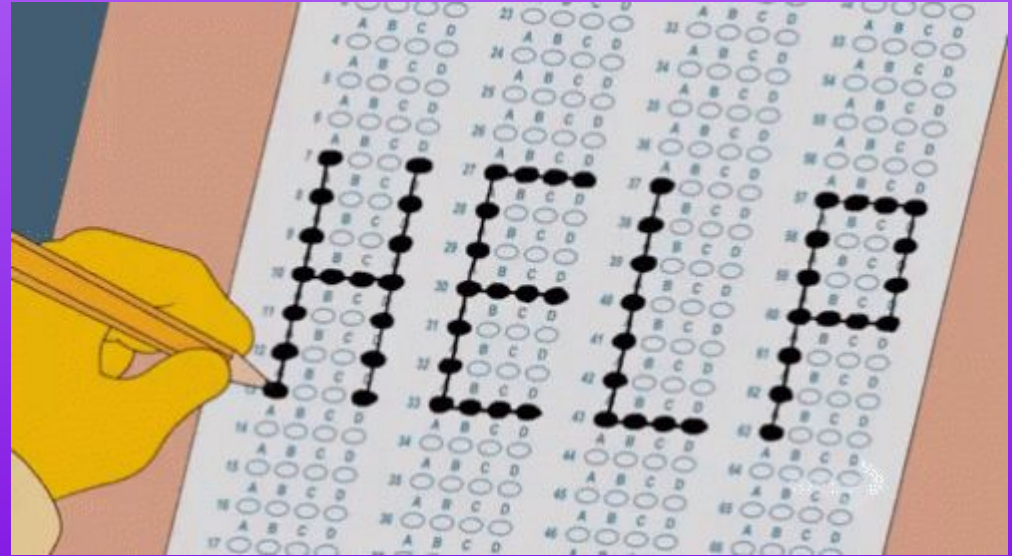


The Problem

Generating expression tests for complex rules is **tedious and time consuming**, taking away from more important feature development

The need for

- Robust expression test coverage
- Relevant test inputs
- Valuable assertions



How it Works

1. **Export** — Pulls the objects in a package from Appian

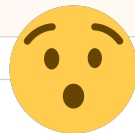
2. **Analyze** — AI reads and understands the SAIL code

3. **Generate Tests** — Jarvis generates test cases based on best practices and rule context

4. **Deploy** — Rebuild, inspect, and redeploy in one command

```
{ } mcp.json  expression-test-generation.md x  jarvis-menu.md M
solutions-os > ai-framework > tools > Jarvis > jarvis-power > steering > expression-test-generation.md > @@ # Expression Test Generation Workflow > @@ ## Deploying Test Cases — Batch Deployme
1  ---
2  inclusion: manual
3  ---
4
5  # Expression Test Generation Workflow
6
7  ## Overview
8
9  This workflow generates test cases in XML format for expression rules found in an exported Appian package. It reads the SAIL definitions from the export, analyzes
used for validation.
10
11 **Common Triggers:**
12 - "Generate test cases for expressions in package {UUID}"
13 - "Create test XML for the expression rules in this export"
14 - "Write test cases for {expression rule name}"
15 - "Generate tests from exported package"
16
17 ---
18
19 ## Quick Path (from existing export ZIP)
20
21 If the user already has an export ZIP in the 'exports/' folder:
22
23 ...
24 1. extract_sail_from_export(export_zip_path="exports/export-uuid.zip")
25 2. [AI analyzes expression rules and generates test case XML]
26 ...
27
28 **If the package contains multiple expression rules**, use the chained rebuild approach (see "Multi-Object Packages" section under Deploying Test Cases).
29
30 ---
31
32 ## Full Path (from package UUID or object name)
33
34 ...
35 1. export_package(package_uuid="UUID")
36 2. get_export_results(export_uuid="...") → downloads ZIP
37 3. extract_sail_from_export(export_zip_path="exports/export-uuid.zip")
38 4. [AI analyzes expression rules and generates test case XML]
39 ...
40
41 **Finding the package UUID:**
42 - If user provides a package URL: extract the base64 ID from the URL path
43 - If user provides an object name: use 'jarvis_search_objects' to find it, then identify the package
44
45 ---
46
47 ## CRITICAL: Check Existing Test Cases Before Modifying
48
```

Example 1: Test Coverage



Look at those
descriptive names!
So much better
than "Test!"

</> RS_CO_UT_formatCurrencyValue

```
1 cast(  
2   'type:[http://www.appian.com/ae/types.  
3   if(  
4     rule!AS_PD_CO_UT_isBlank(  
5       ri!currencyValue  
6     ),  
7     if(  
8       ri!showHyphen,  
9       "",  
10      ""  
11    ),  
12    with(  
13      local!allCurrencies: rule!AS_PD_C  
14      local!currencyCode: rule!AS_PD_CO  
15      nullableValue: ri!currencyCode,  
16      replacementValue: cons!AS_PD_CO  
17    ),  
18    local!symbol: displayvalue(  
19      local!currencyCode,  
20      local!allCurrencies.currencyCod  
21      local!allCurrencies.symbol,  
22      null  
23    ),  
24    local!currencyDisplay: if(  
25      ri!currencyValue = 0,  
26      if(  
27        ri!showDecimal,  
28        "0.00",  
29        "0"  
30      ),  
31      text(  
32        ri!currencyValue,  
33        "
```

Ad Hoc Test

Test Cases (11)

NEW TEST CASE

RUN ALL TESTS

☐	Name	Output	Assertion	Time (ms)	currencyValue	currencyCode	showDecimal	showCurrencyCode
☐	Happy Path - USD with decimal an... DEF...	\$1,234.56 USD	Expression: test!output = "\$1,234.56 USD"	1	1234.56	USD	true	true
☐	Happy Path - USD with decimal, no curre...	\$1,234.56	Expression: test!output = "\$1,234.56"	1	1234.56	USD	true	false
☐	Happy Path - No decimal places, truncates	\$1,234	Expression: test!output = "\$1,234"	1	1234.56	USD	false	false
☐	Happy Path - EUR currency with Euro sy...	€1,000.00 EUR	Expression: test!output = "€1,000.00 EUR"	1	1000	EUR	true	true
☐	Happy Path - GBP currency with Pound s...	£1,000.00 GBP	Expression: test!output = "£1,000.00 GBP"	1	1000	GBP	true	true
☐	Happy Path - JPY currency with Yen symbol	¥1,000.00 JPY	Expression: test!output = "¥1,000.00 JPY"	1	1000	JPY	true	true
☐	Edge Case - Unrecognized currency code,...	1,000.00 XYZ	Expression: test!output = "1,000.00 XYZ"	1	1000	XYZ	true	true
☐	Null Handling - Null value returns empty ...	null	Expression: test!output = ""	1	null	USD	true	true
☐	Null Handling - Null value with showHyp...	-	Expression: test!output = "-"	1	null	USD	true	true
☐	Null Handling - Null currency code default...	\$1,234.56 USD	Expression: test!output = "\$1,234.56 USD"	1	1234.56	null	true	true
☐	Boundary - Zero value displays correctly	\$0.00	Expression: test!output = "\$0.00"	1	0	USD	true	false

11 items

Example 2: Complex Rule Inputs

Edit Test Case

Name
Boundary - 30 sections at max (no error) ☐ Set as default test case

▼ Test Inputs

Rule Input Name	Description	Expression	Value
sections (List of AS_AIDB_T_Section_RecordType)		1 * repeat(C30, recordType[AS_AIDB_T_Section_RecordType C AS_AIDB_T_Sect	List of AS_AIDB_T_Section_RecordType: 30 items AS_AIDB_T_Section_RecordType title: "Title" purpose: ...More

▼ Assertion

Test will pass when

☐ Test completes without errors

☐ Test output matches the asserted output

☒ Assertion expression evaluates to true

Expression

```
1 test!output = false
```

▼ Test Output

Status ☒ Passed

Value ☒ Formatted ☐ Raw ☐ Explore



You no longer have to construct, or copy and paste! I did this for you!

Edit Test Case

Edit Test Case

Name
Happy Path - Status not completed returns no ☒ Set as default test case

▼ Test Inputs

Rule Input Name	Description	Expression	Value
brandingMap (Map)		1 rule!AS_PD_LoadBrandingMap()	Map SiteLogo: 26650 - AS PD Appian Logo.png (Document) SiteFavicon: 26651 - AS PD ProcureSight Enterprise Favico...More
collection (AS_PD_Collection_RecordType)		1 * recordType[AS_PD_Collection_RecordType C AS_PD_Collection_RecordType.c	AS_PD_Collection_RecordType collectionId: 1 modifiedBy: "test.user" collectionEntityMaps: List of AS_PD_Coll...More
refData (List of AS_PD_R_Data_RecordType)		1 * rule!AS_PD_QR_getRefData(returnType: "OBJECT_ARRAY")	List of AS_PD_R_Data_RecordType: 8 items AS_PD_R_Data_RecordType refDataId: 1 refLabel: "Queued" ...More
statusId (Number (Integer))		1 1	1

</> AS_PD_validateDownloadDocumentForDocGeneration

SAVE 🔍 ⚙️ ☰ AG appian

```

1  v a!localVariables(
2      local!aidblimitations: rule!AS_PD_MAP_applimitatio
3      local!hasError: or(
4          a!isNotNullOrEmpty(r!docDownloadResponse.error)
5          a!isNullOrEmpty(r!docDownloadResponse.dataSubse
6          a!isNullOrEmpty(r!docDownloadResponse.dataSubse
7      ),
8      local!downloadedDocs: if(
9          local!hasError,
10         null,
11         rule!AS_PD_CO_UT_castToMapList(
12             value: r!docDownloadResponse.dataSubset.data
13         )
14     ),
15     if(
16         local!hasError,
17         a!localVariables(
18             local!invalidDocumentMapIndex: rule!AS_PD_CO_U
19             records: r!selectedDocumentMaps,
20             field: "appianDocId",
21             value: null,
22             filterType: "MATCH_INDICES"
23         ),
24         a!map(
            
```

Ad Hoc Test | **Test Cases (2)**

NEW TEST CASE | RUN ALL TESTS

☐	Name	Output	Assertion	Time (ms)	docDownloadResponse	selectedDocumentMaps
☒	Error Response DEFAULT	[validDocumentMaps:,documentErrors:[...]	No errors	38	null	[primaryKey:Solar+Farm+RFI+...
☒	Response	[validDocumentMaps:[primaryKey:Solar+...	No errors	5	[dataSubset:[startIndex=1, batchSize=-1, ...]	[primaryKey:Solar+Farm+RFI+...

After:

The screenshot displays the Appian IDE interface. On the left, a code editor shows a test script named `RS_Test_Map_Simple`. The script defines a `rule!AS_PD_MAP_applimitationsForAidb` and includes logic for handling document download responses, including error handling and filtering. On the right, the 'Test Cases (5)' tab is active, showing a table of test results. The table has columns for Name, Output, Assertion, Time (ms), docDownloadResponse, and selectedDocumentMaps. Five test cases are listed, all with a green checkmark indicating they passed. The test cases cover null handling, download errors, missing data subsets, empty data subsets, and all documents having null appianDocId.

	Name	Output	Assertion	Time (ms)	docDownloadResponse	selectedDocumentMaps
☐	Null Handling - Both inputs null DEFAULT	[validDocumentMaps;documentErrors:]	Expression: and(alisNullOrEmpty(testout...	46	null	null
☐	Error Branch - Download error with mixe...	[validDocumentMaps:[fileName:valid.doc...	Expression: and(length(test!output.valid...	3	[error:Download failed]	[fileName:report.pdf,fileType:pdf,appian...
☐	Error Branch - Missing dataSubset	[validDocumentMaps:[fileName:test.pdf,...	Expression: not(!snul(test!output))	2	[error;dataSubset:]	[fileName:test.pdf,fileType:pdf,appianDo...
☐	Error Branch - Empty dataSubset data	[validDocumentMaps:[fileName:test.pdf,...	Expression: not(!snul(test!output))	2	[error;dataSubset:[data:]]	[fileName:test.pdf,fileType:pdf,appianDo...
☐	Error Branch - All docs have null appianD...	[validDocumentMaps;documentErrors:[a...	Expression: and(alisNullOrEmpty(test!out...	3	[error:fail]	[fileName:a.pdf,fileType:pdf,appianDocId...

5 items

Wow, much better coverage!

Impact & What's Next

Impact:

- Accelerated test-case development
- Wide coverage of scenarios: happy path, null, off-by-one, out-of-bounds
- Single prompt → scenarios, scenario names, and assert outputs

Supported today:

- Wide test scenario coverage
- Complex data structures (maps, CDTs, Record Types)
- Seeks dependent rules (e.g. branding, ref data) for test case generation

What's next:

- POC of using extant expression rules in test cases → using utility and helper rules to populate data
- Expand to Interface default test scenarios
- Validate expression tests before deployment



Kiro to FigJam

From spec to user flow diagram in seconds

Overview

1. The problem
2. How it works
3. Demo

The Problem

1. Specs live in markdown, diagrams live in FigJam — they drift apart
2. Drawing user flows manually is tedious and hard to keep in sync
3. No easy way to validate a diagram against the source of truth

How it works

1. Reads a spec and generates user flow diagrams directly in FigJam
2. Keeps diagrams and specs in sync as requirements evolve
3. Spots UX friction points and spec gaps automatically

Demo

Checkpoint Restore

create a user flow diagram based on the writing personas spec in figjam

Kiro

Including Steering Documents `git-workflow.md`

Searched workspace

- Looking for the writing personas spec file to understand the user flows I need to diagram
- Searching for persona-related files in the workspace
- Looking for writing personas in the AI Document Builder feature area
- Looking for the writing personas spec in the AI Document Builder feature folder

Read file(s) `AI Document Builder Background.md`

Now let me look at the writing personas spec more specifically. Let me check if there's a spec file related to writing personas:

Searched workspace

Searching for writing personas content across the workspace

Read file(s) `requirements.md`

Now let me read the POWER.md for the FigJam user flows power to understand how to use it:

Read the full spec and understand the process. Let me now parse the requirements into flow structures and present a plan.

Based on the **Custom Writing Personas** spec, I found **11 requirements**. Here's the list:

- Req 1:** Admin Creates a Writing Persona — Admin creates personas via AI generation, document extraction, or manual entry
- Req 2:** Admin Edits and Deletes a Writing Persona — Admin modifies or removes personas from the library
- Req 3:** Browse and Search the Persona Library — Document Editor browses/searches available personas
- Req 4:** Admin Assigns a Persona to a Template — Admin links a persona to a template for inheritance
- Req 5:** Select a Persona for Document Generation — Document Editor picks a persona before generating a draft
- Req 6:** Apply and Override a Persona During Writing Assistant Refinement — Document Editor uses/overrides persona during editing
- Req 7:** User Default Persona — Document Editor sets a personal default persona
- Req 8:** Admin Manages Compliance Rules Within a Persona — Admin adds/edits/removes compliance rules
- Req 9:** Compliance Monitor — System evaluates content against active persona's rules
- Req 10:** Default Persona Behavior — System fallback when no persona is active
- Req 11:** Persona Serialization and Portability — Admin exports/imports personas as JSON

Which requirements should I diagram? All 11, a specific subset, or a range?

Also — since this will be a FigJam file, I'll create a new one for you. Let me know your preference and I'll present a mermaid preview of each flow for your approval before writing anything to FigJam.

Checkpoint Restore

start with requirement 1. use <https://www.figma.com/board/mnoNul4HGZJyxjNPgGzwGk/User-flows?node-id=640-4498>

Kiro

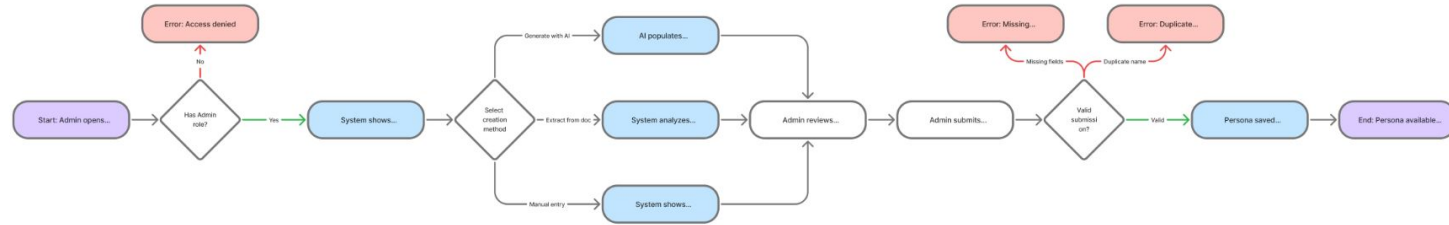
Here's the planned flow for **Requirement 1: Admin Creates a Writing Persona**:

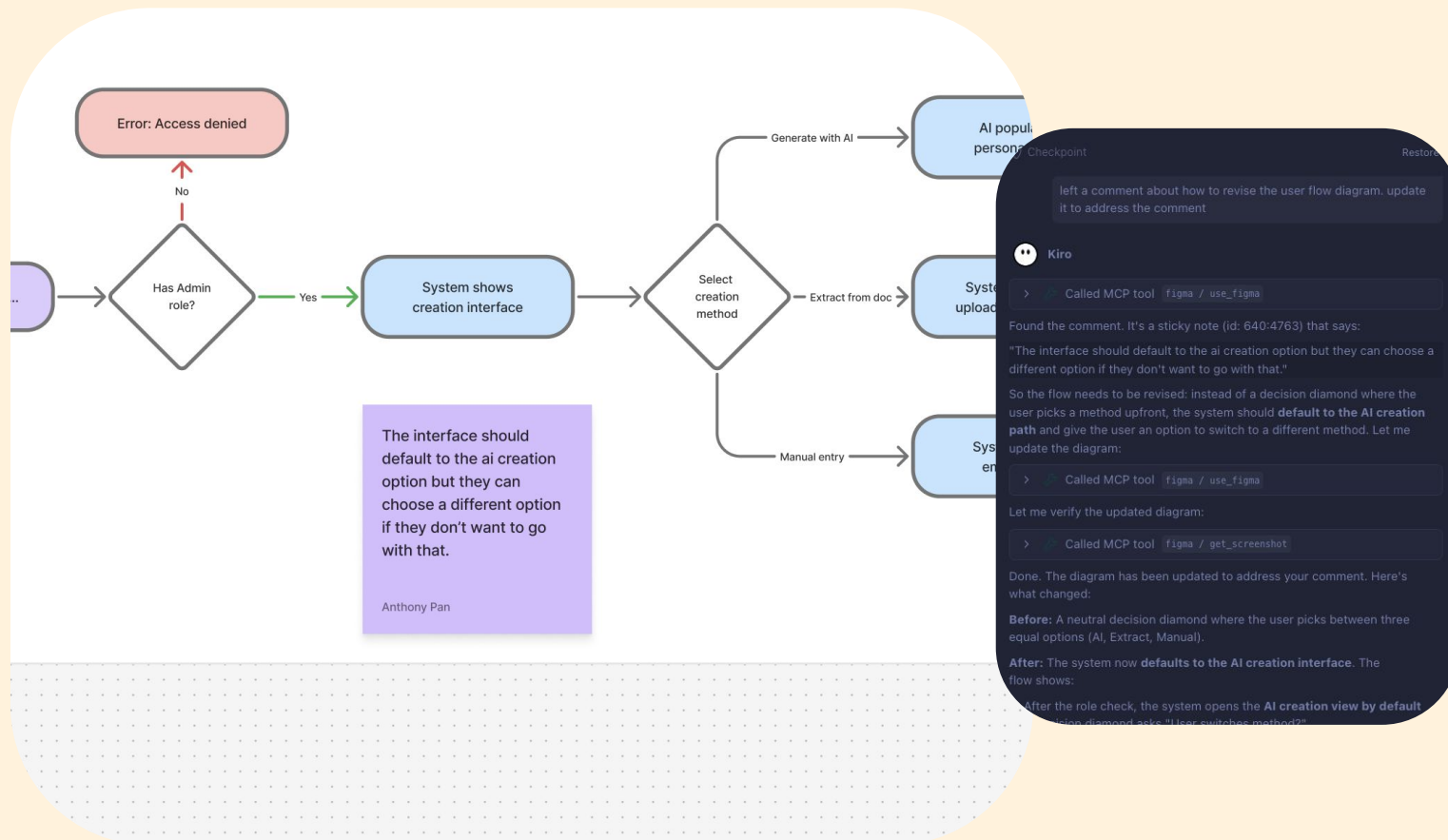
```
graph LR
    A["Start: Admin opens Persona Manager"] --> B["Has Admin role?"]
    B -->|No| C["Error: Access denied"]
    B -->|Yes| D["System shows creation interface"]
    D --> E["Select creation method"]
    E -->|Generate with AI| F["AI populates persona fields"]
    E -->|Extract from document| G["System analyzes document"]
    E -->|Manually fill in| H["System shows empty form"]
    F --> I["Admin reviews populated fields"]
    G --> I
    H --> I
    I --> J["Admin submits persona"]
    J --> K["Valid submission?"]
    K -->|Name is duplicate| L["Error: Name conflict"]
    K -->|Missing required fields| M["Error: Missing fields"]
    K -->|Valid| N["Persona saved to Library"]
    N --> O["End: Persona available in Library"]
```

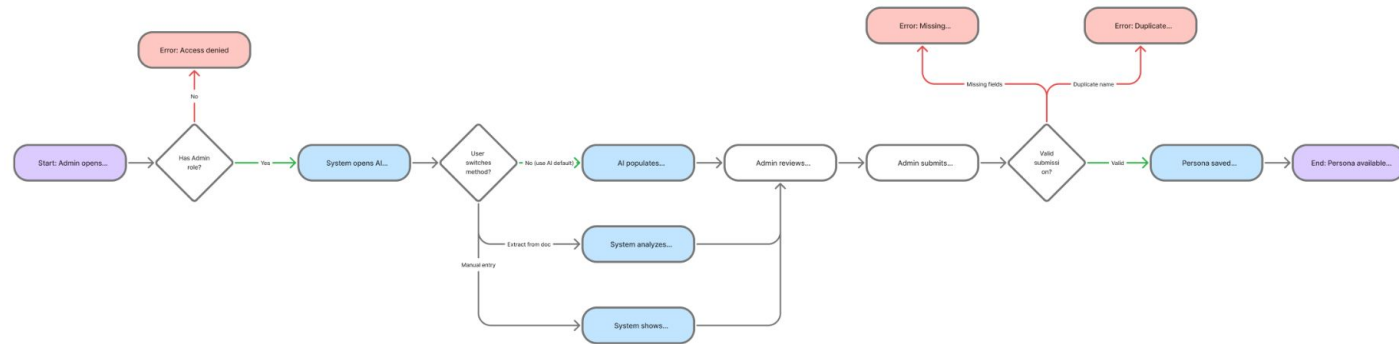
Flow summary:

Actor: Admin

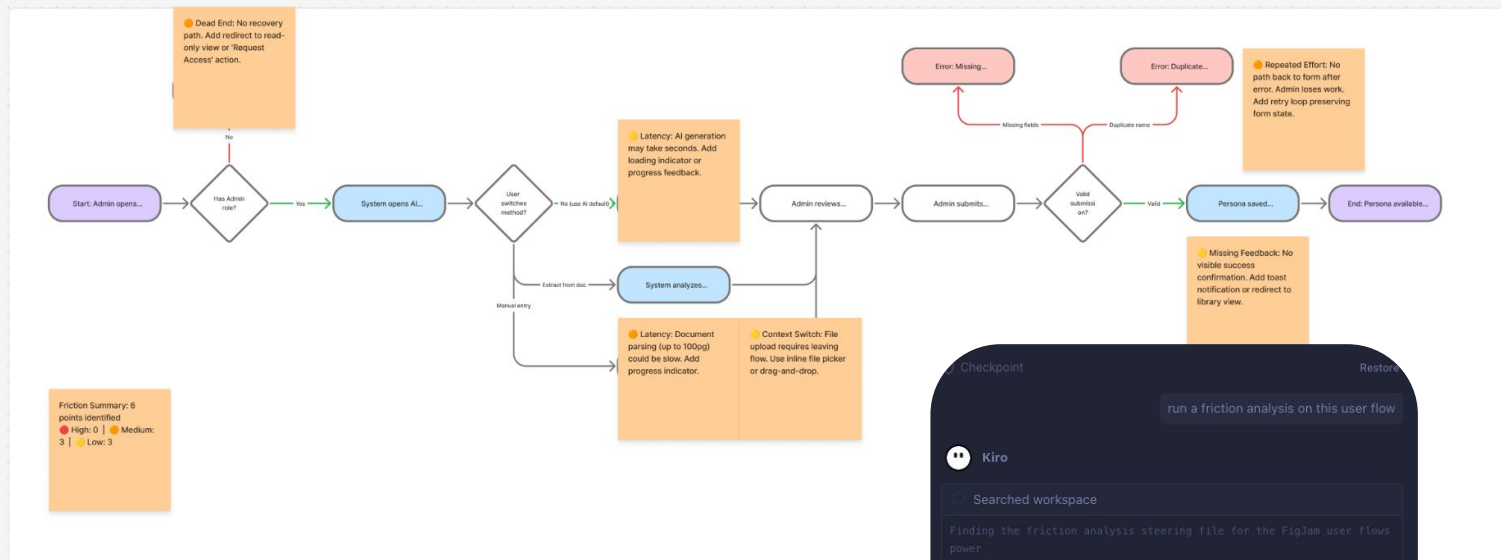
Admin opens the Persona Manager creation interface.

Req 1: Admin Creates a Writing Persona



Req 1: Admin Creates a Writing Persona

Req 1: Admin Creates a Writing Persona



Checkpoint Restore

run a friction analysis on this user flow

Kiro

Searched workspace

Finding the friction analysis steering file for the FigJam user flows power

Read file(s) [action-friction-analysis.md](#)

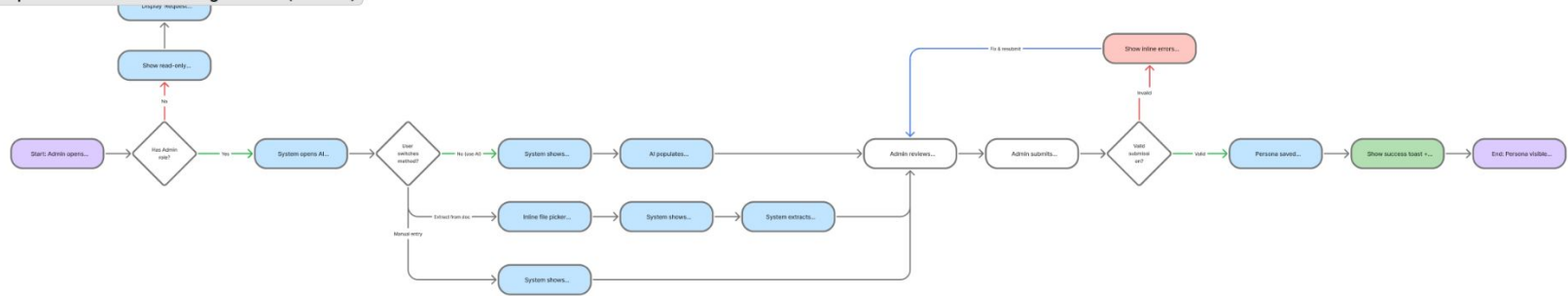
Here's my friction analysis of the **Req 1: Admin Creates a Writing Persona** flow:

Friction Analysis — Req 1: Admin Creates a Writing Persona

● **Friction Point 1: Dead End — "Error: Access denied"**

Location: After "Has Admin role?" → No path

Issue: The error node is a terminal dead end. The non-Admin user is

Req 1: Admin Creates a Writing Persona (Revised)

update the spec to reflect the new user flow

 Kiro

I see you're working on a task that's optimized for spec sessions. Would you like to start a dedicated spec session for the best experience?

No

Read file(s) requirements.md 27 - 56

Accepted edits to requirements.md

The spec has been updated. Here's a summary of the changes to Requirement 1:

Friction Addressed	Spec Change
FP1: Dead end (access denied)	AC #2 now specifies non-Admins see a read-only Persona Library + "Request Admin Access" link instead of just being blocked
FP2: AI latency	AC #4 now requires a progress indicator during AI generation
FP3: Doc parsing latency	AC #5 now requires a progress indicator during document analysis
FP4: Repeated effort (validation)	AC #9 consolidated duplicate name + missing fields into one criterion requiring inline errors with form state preserved for resubmission
FP5: Missing feedback	New AC #11 requires a success toast + redirect to the Persona Library

Thanks!

Jarvis SAIL Canvas



Soma Sundaram
Lead Product Engineer



Govind
User Experience
Designer

Jarvis SAIL Canvas

What it is: An AI-assisted design workspace that bridges Appian's SAIL interfaces with interactive React prototypes — enabling visual iteration on enterprise UI without touching the live environment

Core Capabilities

Import

Pulls live Appian interfaces via Jarvis MCP, resolving all dependencies

State Exploration

Maps every state dimension and renders them with a toggle panel

Natural Language

Iterate with prompts like "Add a KPI card here" — modifies preview with diffs

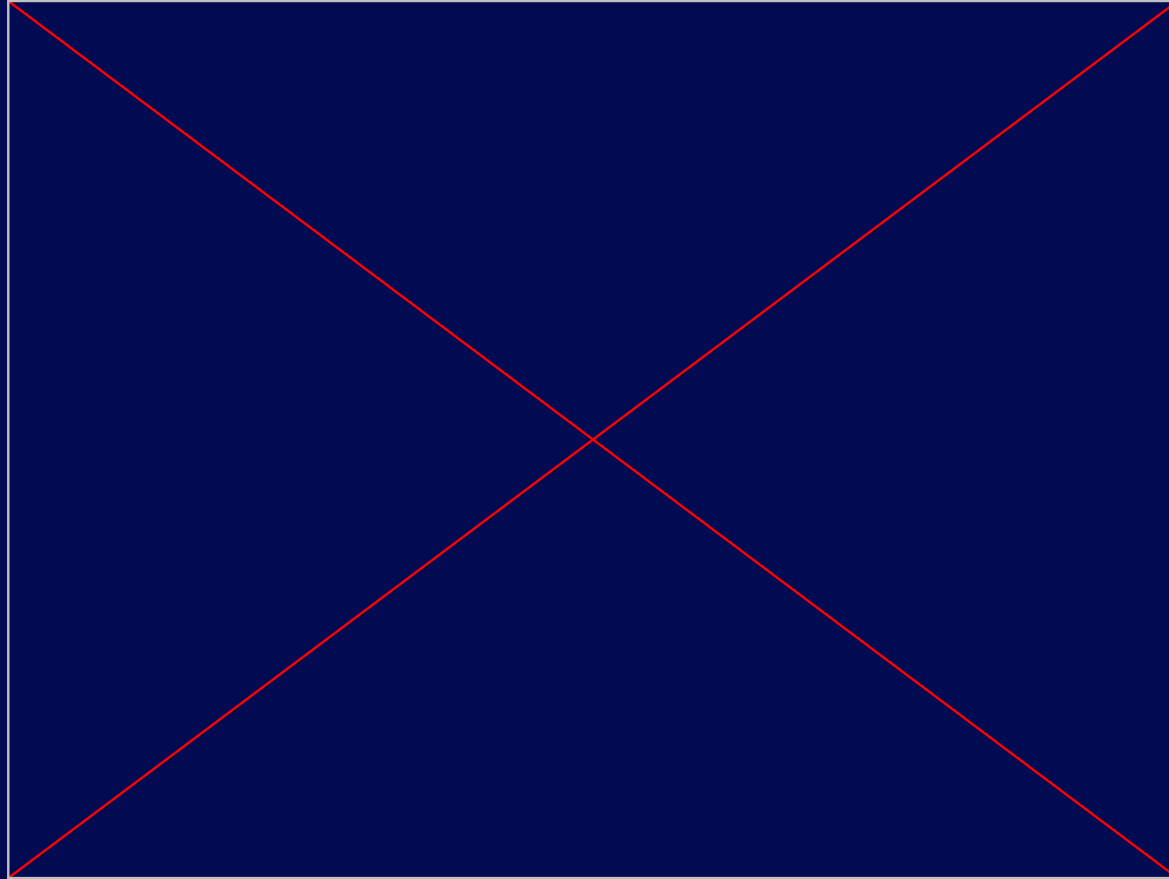
Validated SAIL Export

Generates copy-paste-ready SAIL code that passes evaluator

Record Action Flows

Clickable buttons rendering full form walkthroughs in modals

Jarvis SAIL Canvas



Jarvis SAIL Canvas

Appian Environment

The screenshot displays the Appian Jarvis SAIL Canvas interface for the 'Testing 009' opportunity. The left sidebar contains navigation links: Vendor Management, Home, Opportunities (selected), Contracts, Messages (1), and Vendors. The main content area is titled 'Testing 009' and includes tabs for Summary, Vendors (0), Description and Instructions, Questions, and Revision History. The 'Summary' tab is active, showing details such as Type (Sources Sought), Opportunity Stage (Awaiting Unsealing), Response Due (Nov 22, 2024 • 4:00 AM HST), and Questions Due (Nov 23, 2024 • 4:00 AM AST). Below these are sections for Updates, Response Overview, and Attachments. The Updates section lists several amendments, including 'Amendment w | 322' and 'Amendment test | test'. The Response Overview section shows a table with columns for Interested (2 Vendors), In Progress (0 Vendor), Submitted (0 Vendor), and Clarification Re... (0 Vendor). The Attachments section indicates 'No attachments available yet'.

Local SAIL Canvas Environment

The screenshot displays the Appian Jarvis SAIL Canvas interface for the 'OPP-2026-0042 | IT Infrastructure Modernization Services' opportunity. The left sidebar contains navigation links: Vendor Management, Home, Opportunities (selected), Contracts, Messages (1), and Vendors. The main content area is titled 'OPP-2026-0042 | IT Infrastructure Modernization Services' and includes tabs for Summary, Vendor Proposals, Questions, Vendors, Description & Instructions, and Amendment Changes. The 'Summary' tab is active, showing details such as Type (RFP), Opportunity Stage (Pre-Award), Response Due (Jun 15, 2026 • 5:00 PM ET), and Questions Due (Jun 01, 2026 • 5:00 PM ET). Below these are sections for Updates, Response Overview, and Attachments. The Updates section lists several amendments, including 'Amendment 003 | Response Due Date Extended' and 'Amendment 002 | Updated SOW Section 3.2'. The Response Overview section shows a table with columns for Submitted (4 Vendors), Resubmission Req... (1 Vendor), Interested (7 Vendors), and In Progress (3 Vendors). The Attachments section lists several documents, including 'SOW_IT_Modernization_v3.pdf', 'Evaluation_Criteria.pdf', and 'Pricing_Template.xlsx'.



Jarvis SAIL Canvas

Thank You

[Link to Repo](#)



appian

The T.I.M.E. Framework

Structure that speaks to AI

Taking
Ideas
to
Market
Expedientiously



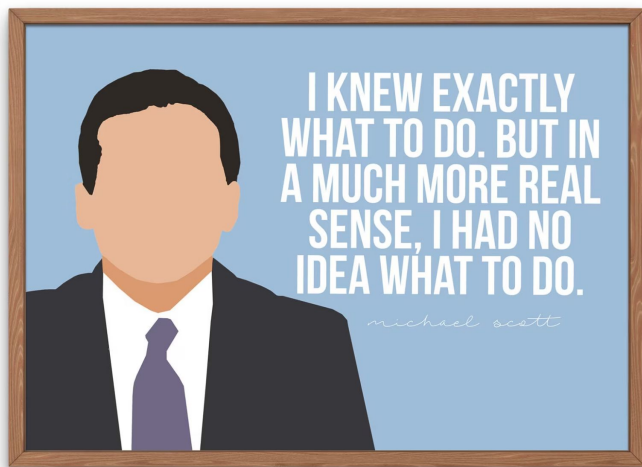
Problem Statement

Even with AI, we're still bleeding time:

- Swivel chairing between AI tools
- Manually asking AI to do work
- Moving artifacts between systems
- Re-explaining or manually injecting context
- Debating what system is the source of truth



A non-technical person (me) wanders into kiro



Structure as a Signal

AI unlocks a huge amount of efficiencies because of its ability to navigate unstructured environments, however, over-reliance on this can undercut it's abilities and sow chaos.

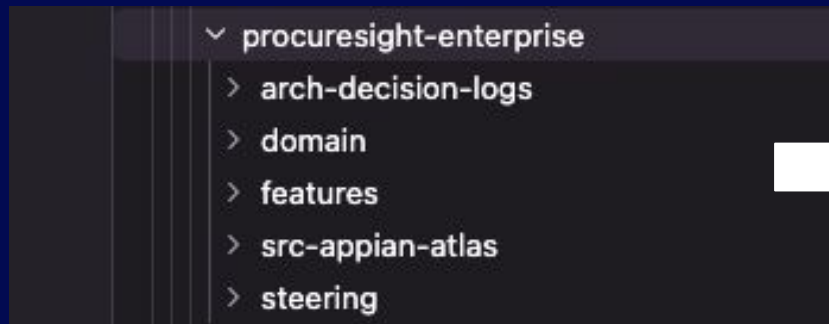
A Kiro Repo Without T.I.M.E.

- Organized by outputs
- Easy to navigate for the tech wizards
- Hard to infer state
- AI must be triggered by the user
- Context is used ad-hoc

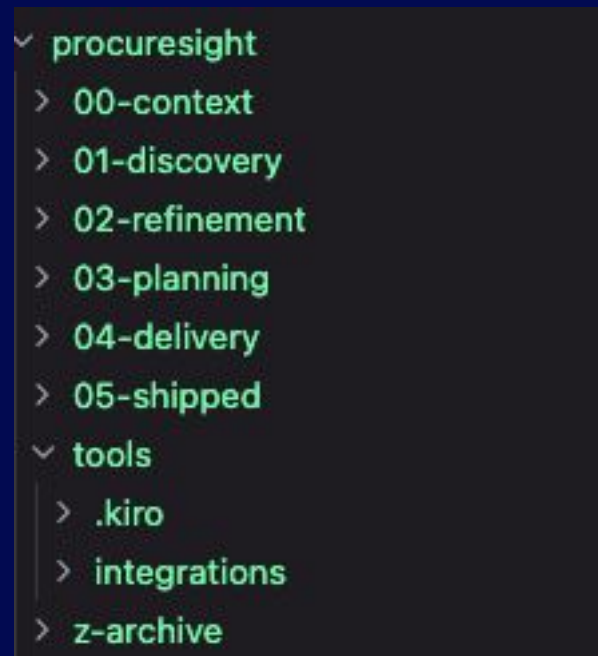
A Kiro Repo With T.I.M.E.

- Organized by (SDLC) outcomes
- Easy to navigate for all disciplines
- Clear understanding of state
- The structure triggers AI
- Context accumulates

Without T.I.M.E.



With T.I.M.E.



Expanded

- ▼ procuresight
 - ▼ 00-context
 - > engineering
 - > product
 - > ux
 - ▼ 01-discovery
 - > feedback
 - > ideas
 - > research
 - ▼ 02-refinement
 - > feature-cards
 - > prototypes
 - > specs
 - ▼ 03-planning
 - > a-committed
 - > b-story-creation
 - > c-ready-for-dev
 - ▼ 04-delivery
 - > active-development
 - > quality-and-testing
 - > sprint-reviews
 - ▼ 05-shipped
 - > enablement
 - > releases / release-notes

How Ideas Progress

Every idea moves through progressive phases, structured as folders. Where something lives tells you its maturity, its readiness, and who should be paying attention to it.

Folder	What Happens Here
 00 — Context	The ground truth. Vision, design, architecture, current features.
 01 — Discovery	Raw ideas, feedback, and research. The intake funnel.
 02 — Refinement	Ideas being shaped into feature specs and prototypes.
 03 — Planning	Committed work, prioritized and ready to be broken down.
 04 — Delivery	Active development and quality assurance.
 05 — Shipped	The record of what made it to market.

Signal Unlocks Automation

When lifecycle stages are explicit, AI doesn't wait for you to ask — it acts on transitions.

1. **Discovery:** *When a new idea is added...* AI scans for duplicates and flags cross-suite dependencies.
2. **Refinement:** *When moving from idea to feature...* AI expands the idea using context, generates the feature card, drafts the spec document, and kick off a lightweight prototype.
3. **Planning:** *When a feature is committed...* AI breaks down the feature, creates tickets for review, and relates it to a release.
4. **Delivery:** *When a feature moves to active dev...* AI generates your sprint review slide text and updates the release status.
5. **Shipped:** *When a feature is shipped...* AI writes release notes, posts, and forum updates.

"AI does better in a
process" - Matt Calkins
- Ben Lloyd



Perf-Profiler

Automated node level performance profiling for Appian applications

Problem Statement:

- Manual instrumentation of process models and reporting setup to capture node-level metrics as part of performance testing is a repetitive, release-by-release effort that scales poorly and can take hours to a full day for larger applications.





How it works:

- Upload or Refer an Appian export zip → get a fully instrumented zip in seconds
- What it does automatically:
 - Injects node Metrics process variable (Text?list) into every process model
 - Adds standardized JSON output expressions on all eligible nodes
 - Updates version UUIDs and history for traceability
 - Merges 10 pre-built PERF PRO reporting objects (Web API, Integration, Expression Rules, Record Type)
 - Generates import customization (.properties) file
- Zero overhead during testing — only stores JSON strings in a PV, no DB writes, no extra nodes
- Idempotent — safe to re-run; already-instrumented nodes are skipped, version just bumps

Impact:

- Performance test setup: Hour/Day-long effort → Minutes
- Eliminates repeated instrumentation for every release cycle
- Enables complete node-level profiling across all process models
- Accelerates bottleneck identification during performance & concurrent testing

Where It Fits:

- Performance Testing and concurrent Testing scenarios
- Any application requiring deep node-level performance insights

Scalability & Reusability:

- Generic solution - not tied to a specific application
- Works across any Appian solution/export
- Easily reusable for multiple releases and environments
- Plug and play for any performance testing cycle



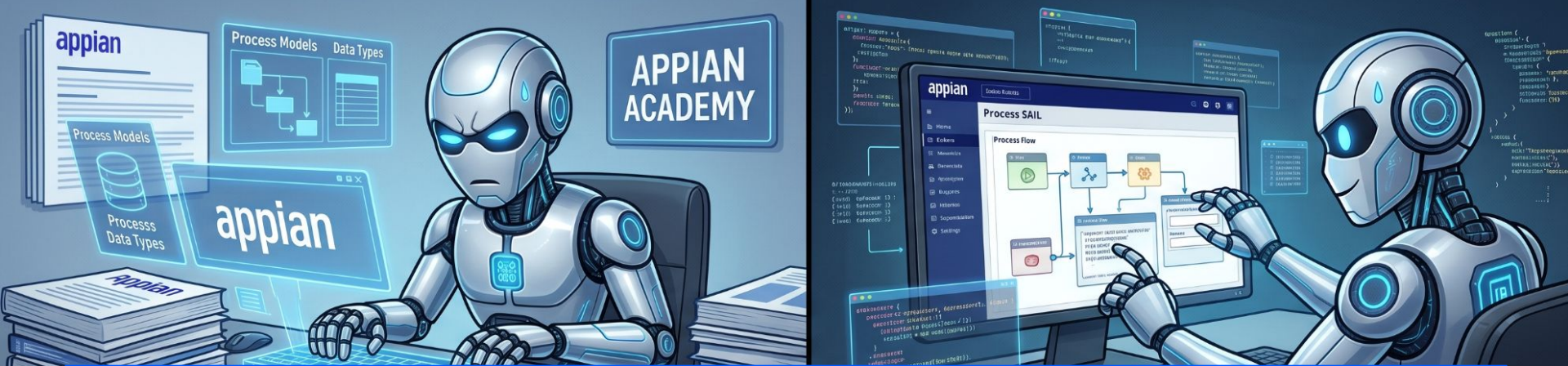
Perf-Profiler

Automated node level performance profiling for Appian applications

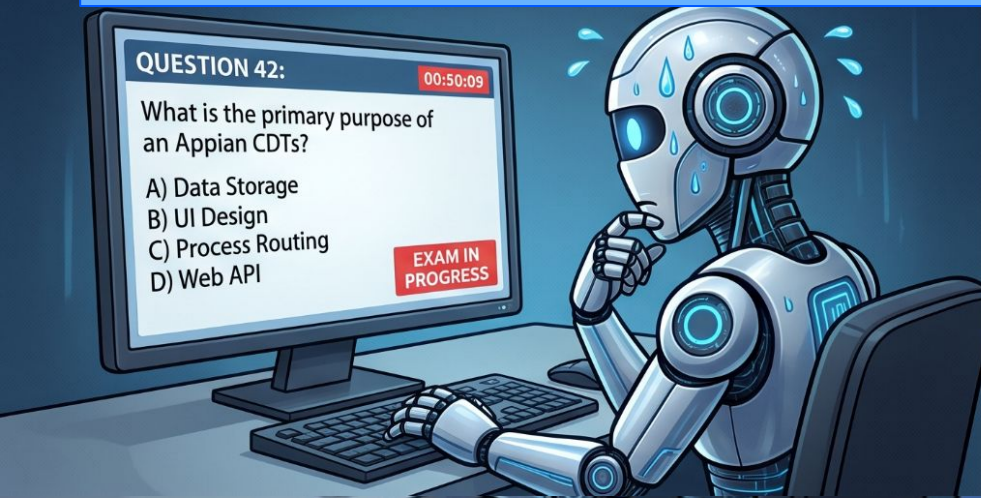
Problem Statement:

- Manual instrumentation of process models and reporting setup to capture node-level metrics as part of performance testing is a repetitive, release-by-release effort that scales poorly and can take hours to a full day for larger applications.





LCP APIs - Accelerating Appian Development



Meet the team



Shaik Ayisha



Eric



Hunter



Karan



Saurabh

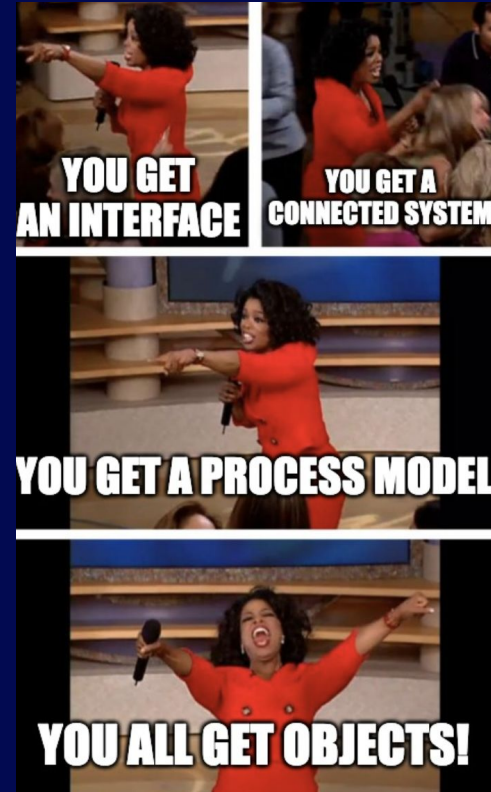


a|migo



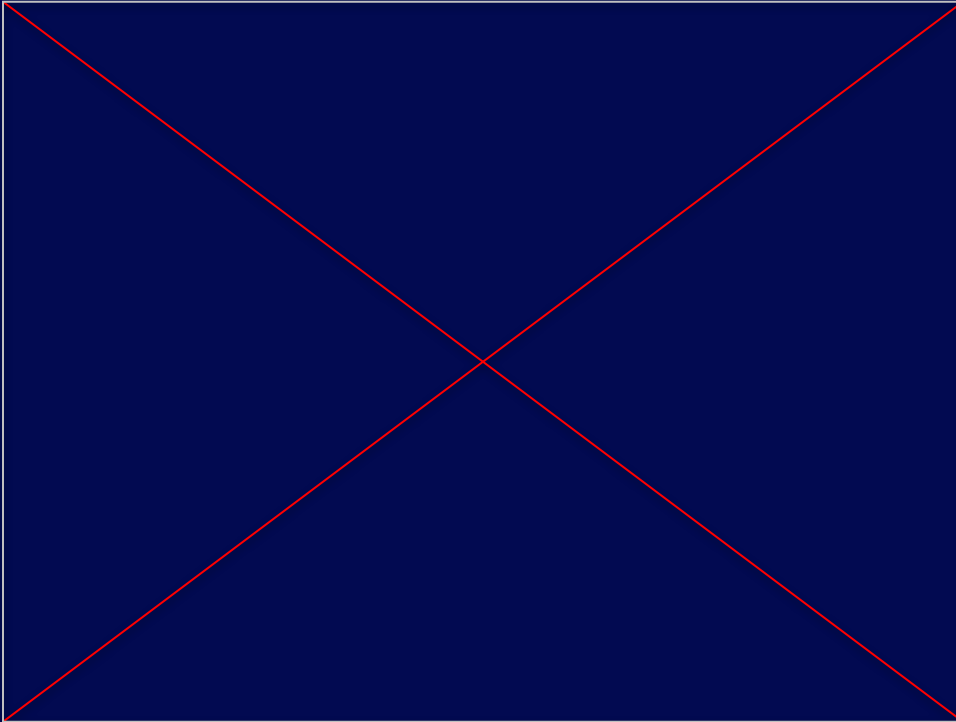
What is it?

- Supports 130+ Appian operations: interfaces, expression rules, constants, process models, connected systems, you name it.
- The data model workflow is the first fully polished pipeline.
- The foundation is there to automate other tasks and workflows to make it predictable, consistent and usable



Data Model Workflow

- Creates records/data model from a Google Sheet



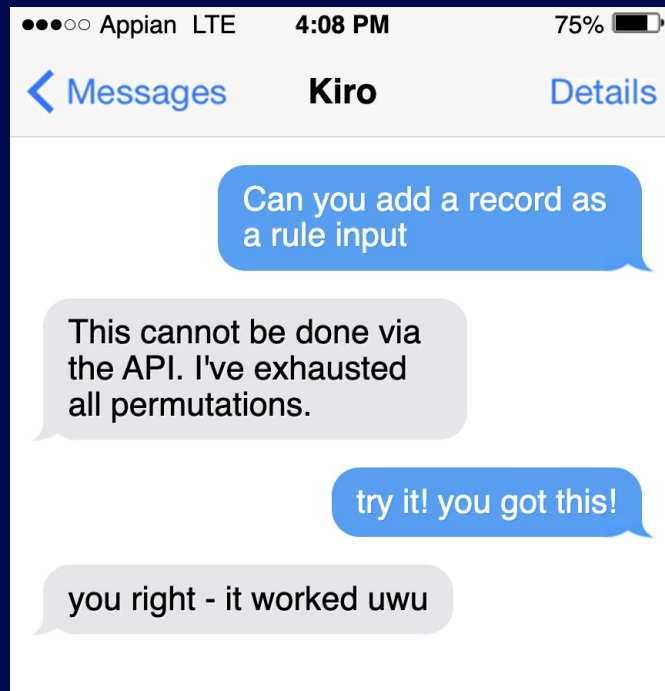
Other potential workflows

- Mockups to SAIL objects
 - Issues with certain data types for rule input
 - Inconsistent due to API quirks
 - Be conscious of updating
- Create query records and other patterned expression rules
 - Behavioral inconsistencies with record references
 - Can generate rules with simple logic
 - Able to reuse generated rules in other interfaces/objects
- Is able to get 75-80% of the way there



AI Quirks & Best Practices

- It's confident but not always right always review the output
- It works best with clear, specific instructions
 - "Create data model from this sheet" beats "do the thing"
- Use a CI site for development
 - You break it, you own it
 - Use a different user
- Consistency challenge
 - Produce consistent expected output
- You can convince AI to do things for you



Impact & What's next

Impact

- Major time savings for developers
- Increased accuracy while setting up data models

What's Next

- Support PostgreSQL generation
- Further exploration of the LCP APIs' capabilities
- Refinement of the other workflows mentioned previously



Fortnightly Report Assistant



Josh



Rob



Saravana

Problem Statement

Every fortnight, TDMs spend **30-60 minutes** per team manually compiling metrics for team health reviews.

Fragmented Data Sources

AWS
Quicksight

Jira
Atlassian

Home
(Eng Delivery)

The Result: More time spent pulling data, less time analyzing it.



Action Plan



Help Needed



Efficiency Metrics (Contingent on working metrics either in EDS or QuickSight)			
Metric	Target	Result	Notes
Tech Debt %	10%	/	
Unplanned work %	<10%	/	
Unknown Work %	0%	/	
Story Cycle Time Average	<5	/	
Last Sprint Commitment Rollover	<10%		
Release Plan/Actual	<10% Behind	/	
Release gates due / warnings	1 / 1	/	
Release metadata updated?	Yes		

AI-Powered SLDC

- Latest Experiments <DATE>
 - Goals & Outcomes
 - Experiments Tracker link
- AI in Daily Work - Team Progress:
 - Bright Spots
 - Frictions

Delivery Milestones Audit

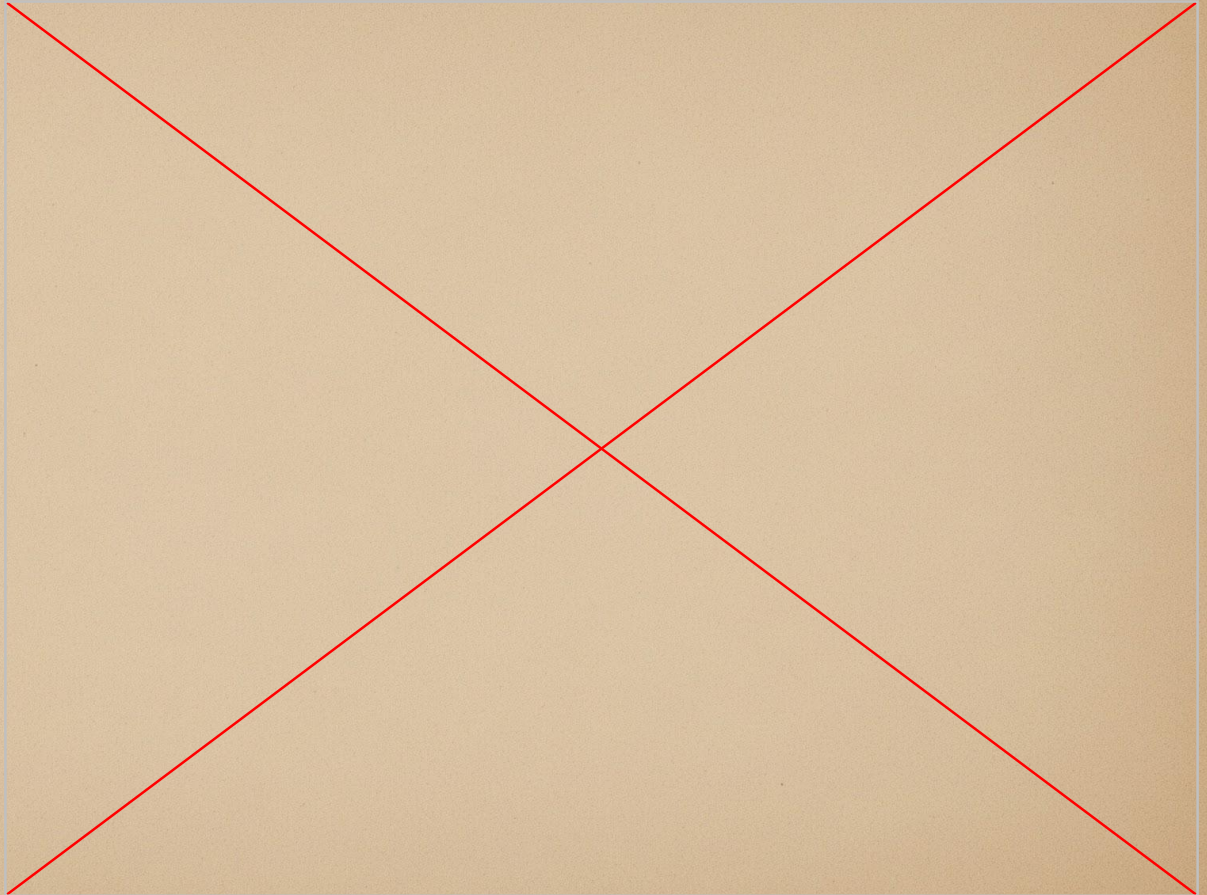
- Last Audit: <DATE>
- Findings (Delivery OKRs & Timeline updated? :
 -

What We Built

One command →
formatted report
inserted in minutes

Python performs the
calculations — no
hallucinations

Color-coded cells
(**green**/**yellow**/**red**)
give instant
health-at-a-glance



Impact & Future Direction

Impact

156 Hours Saved Annually 🌴

8 teams (GPS only) × 26 fortnights × 45 min saved vs. manual assembly.

- Consistent metrics across all teams using standardized formulas and thresholds.
- Shift from data entry to continuous improvement: Managers focus on the "so what."
- Scalable onboarding: New teams can be fully set up in under a minute.

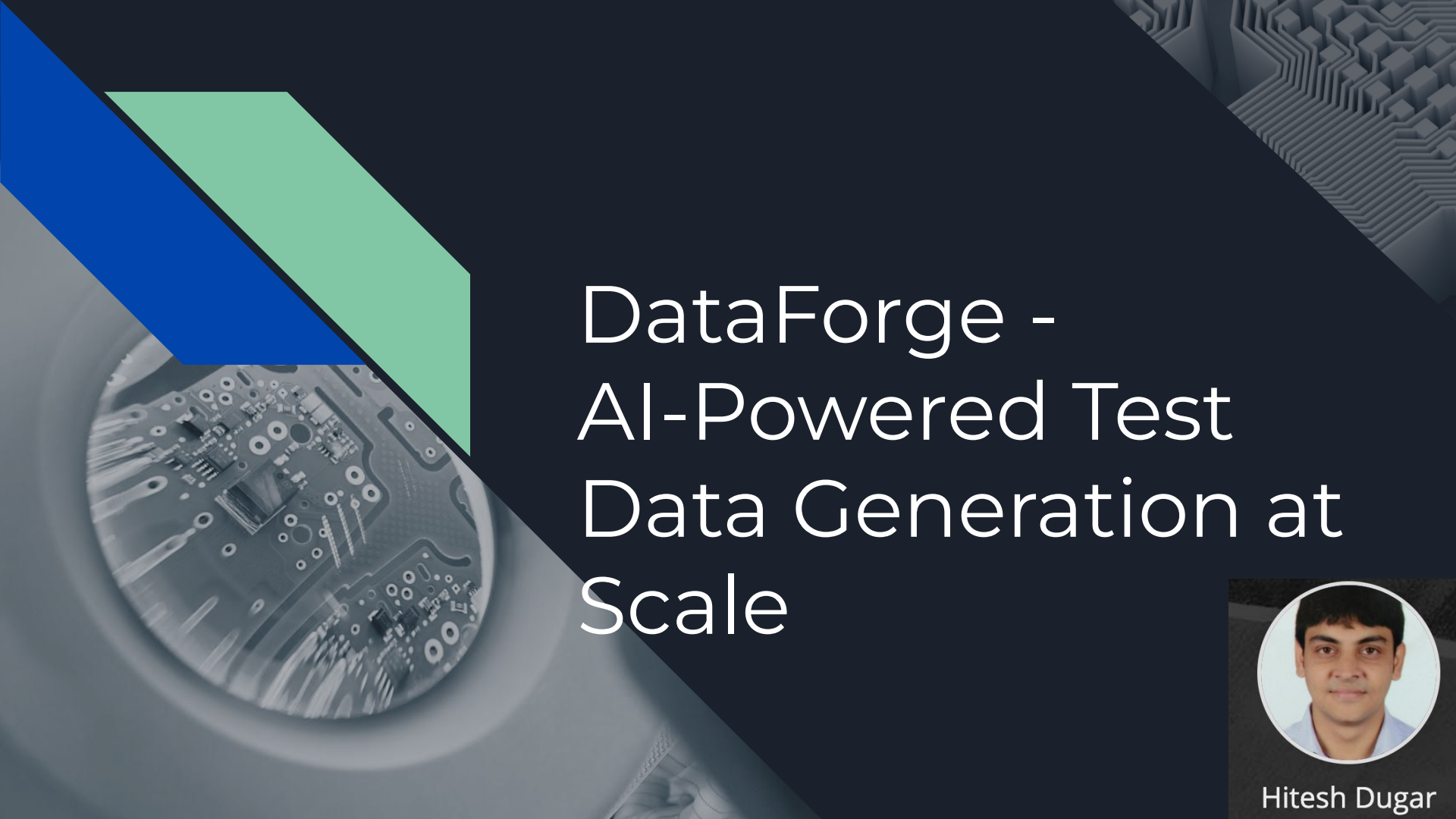
Future Direction

- Onboard other Solutions TDMs
- Integrate with Home MCP
- Build on AI Insights

Help Needed

-

Efficiency Metrics (Generated by fortnightly-report skill)			
Metric	Target	Result	Notes
Tech Debt %	10%	18.2% Trailing 3 months / 11.1% last sprint	
Unplanned/Support work %	<10%	0% Trailing 3 months / 0% last sprint	



DataForge - AI-Powered Test Data Generation at Scale



Hitesh Dugar

The Problem

- Requires deep knowledge of the schema — FK relationships, generation order, platform quirks - making data setup developer-dependent and creating a bottleneck
- Generating data at the volume needed for performance testing is time-consuming and repetitive for different scales
- JSON configs are static - they don't adapt when the schema changes
- Platform-specific quirks can silently invalidate an entire test run without any error message



The Idea

- DataForge MCP server reads the schema, infers what data each column needs, and the AI agent orchestrates the full workflow through conversation
- Anyone can generate data at any scale through plain English conversation
- Define ratios once, scale proportionally — 10x for dev, 100x for load testing, 1000x for stress testing
- Reusable easy to read config per application — first setup captures the knowledge, every future run is a single command regardless of who runs it



What's Been Achieved

- Automatically introspects schema — tables, columns, types, foreign keys, dependencies - no manual mapping
- Infers appropriate generators from column names and types through pattern matching
- Generates in correct dependency order with realistic distributions
- Bulk insert using native database mechanisms to minimize setup time
- Validated on a real application (Clause Automation on Oracle RDS)



Checkpoint

Restore

I just installed the dataforge power and want to use it.



Kiro



Including Steering Documents

AGENTS.md

AGENTS.md

sail-tests.md

sail.md

searching.md

git-workflow.md

google-workspace-cli.md

Let me activate the dataforge power to understand what it does and how to get started.



Activated Kiro power dataforge

Now let me also check if there's a steering file with onboarding/usage instructions:



Activated Steering

dataforge-usage.md

Working

Cancel

Follow



Ask a question or describe a task...

#



Auto



Autopilot



Autocomplete



Report issue





What's Next

- Integrate with Atlas to pull valid reference values, business rules, and application context — making generated data behave like production without manual configuration
- Offline mode — generate SQL scripts and CSVs from DDL files for environments where direct database access isn't feasible and mechanism to load automatically rather than manual
- Pre-built easy to read configurations per Appian solution so data generation becomes a user friendly action
- Scale tiers — standardized dataset sizes (S/M/L/XL) per solution for repeatable performance benchmarks

Local*-IDE Development

A speedrun of somewhat local development flows
with Kiro IDE



William Ingold

*Ehh it *can* be 95% local, but IDE development doesn't depend on it

Local* Development - What that means

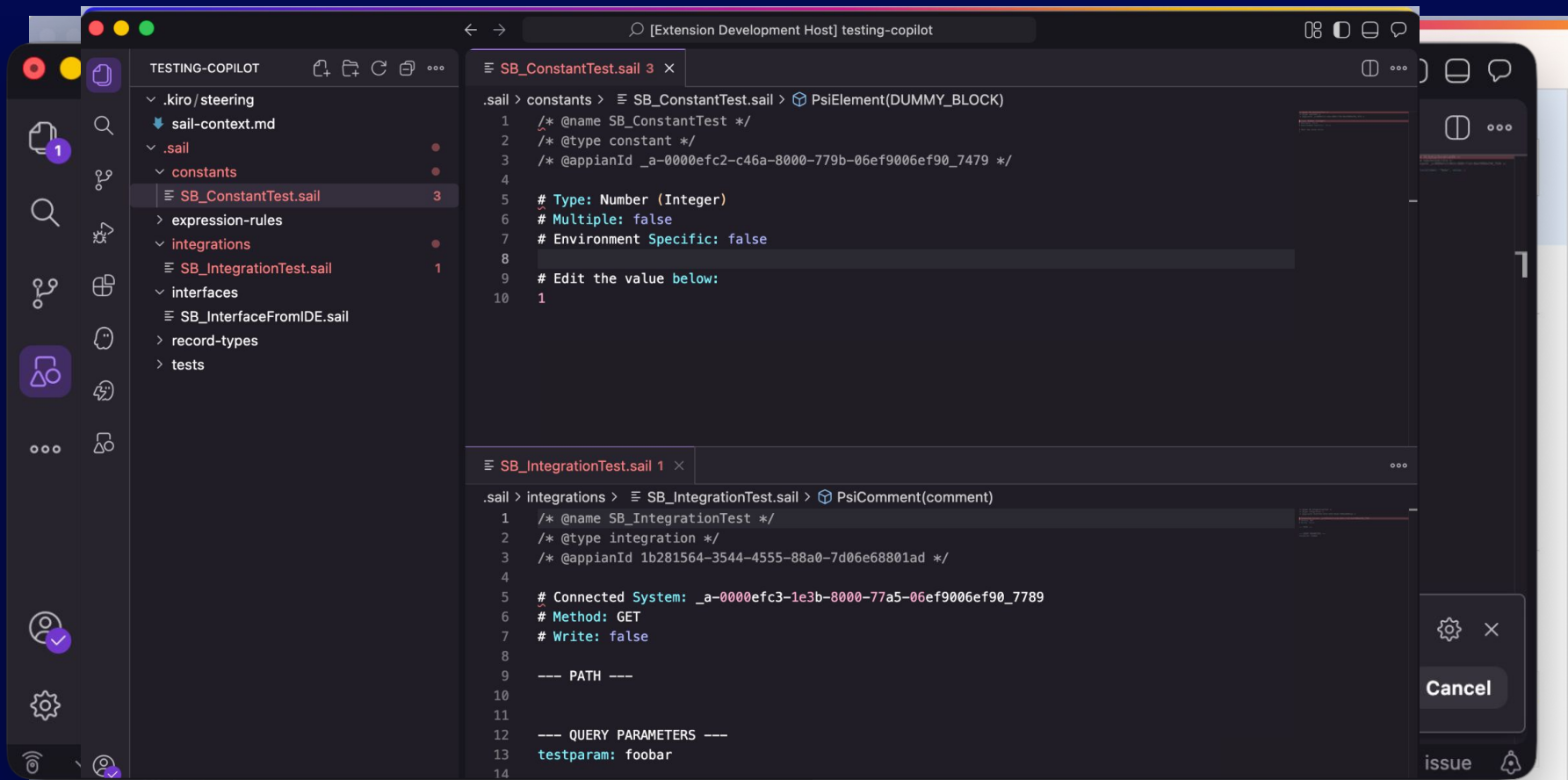
- Typical: Use a “large” EC2 instance for *all* docker services
- Now: All but one* service *can* run locally
 - You get an instance, you get an instance, everyone gets one!
- Pro: Potentially finer control over environment & debugging ability
 - Pro: Smaller, cheaper, cloud for product engineers
- Con: Slower for startup (though that’s rarer now)
 - Con: Makes merging data more difficult
 - Con: Setup is a real pain (for now)
- Neutral: You can run any version of Appian you want cheaper/faster

*Shakes fist at service-manager (K engines)

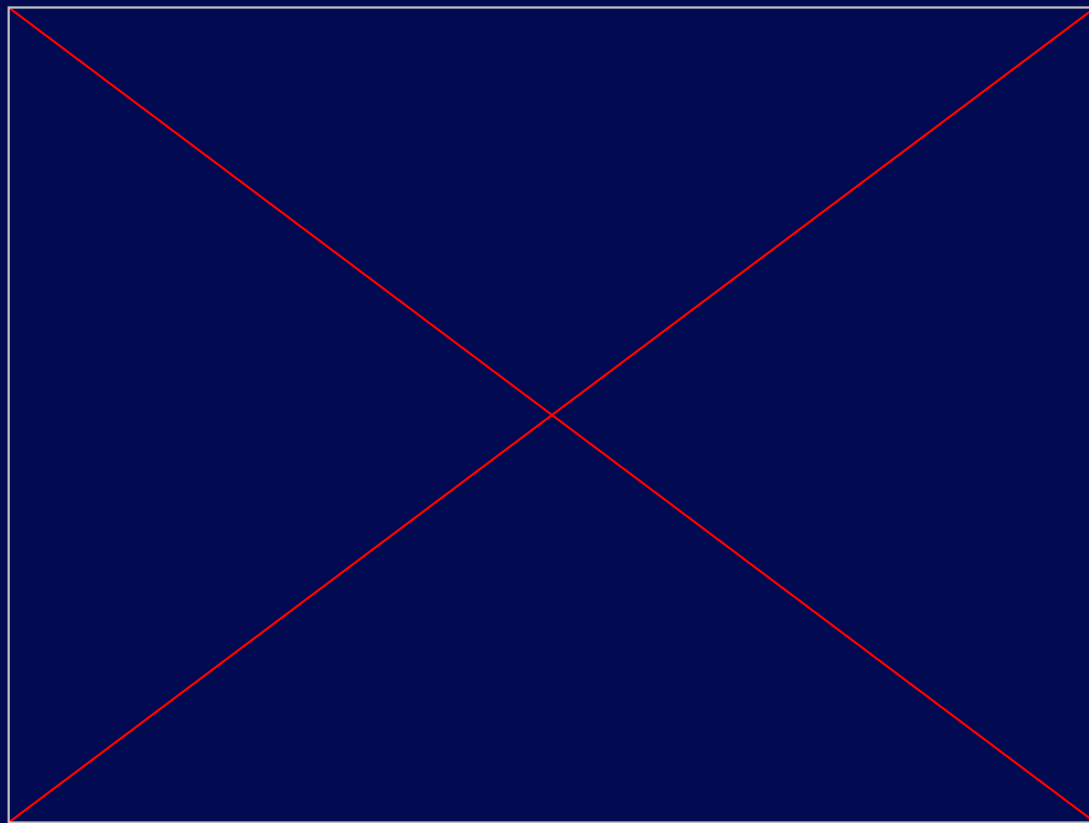
“Local” Development - Power of IDE’s

- Your own IDE (VSCode or Kiro for now)
 - Your key mappings, colorscheme, extension power
 - You get Kiro editing for free
- Extension power!
 - Treeview of the objects you’re working on
 - Code snippets
 - Error highlighting
 - Inline documentation
 - Autocomplete with hints
 - Textual formats for objects (integrations, constants, more?)
 - Creation of tools that would be difficult to embed in Appian
 - Tools we can do faster with scripts than AI interactions with the site
 - <Insert your extension idea here>
- “Local” because this just needs the LCP API enabled on your site
 - *Can* work for remote sites too

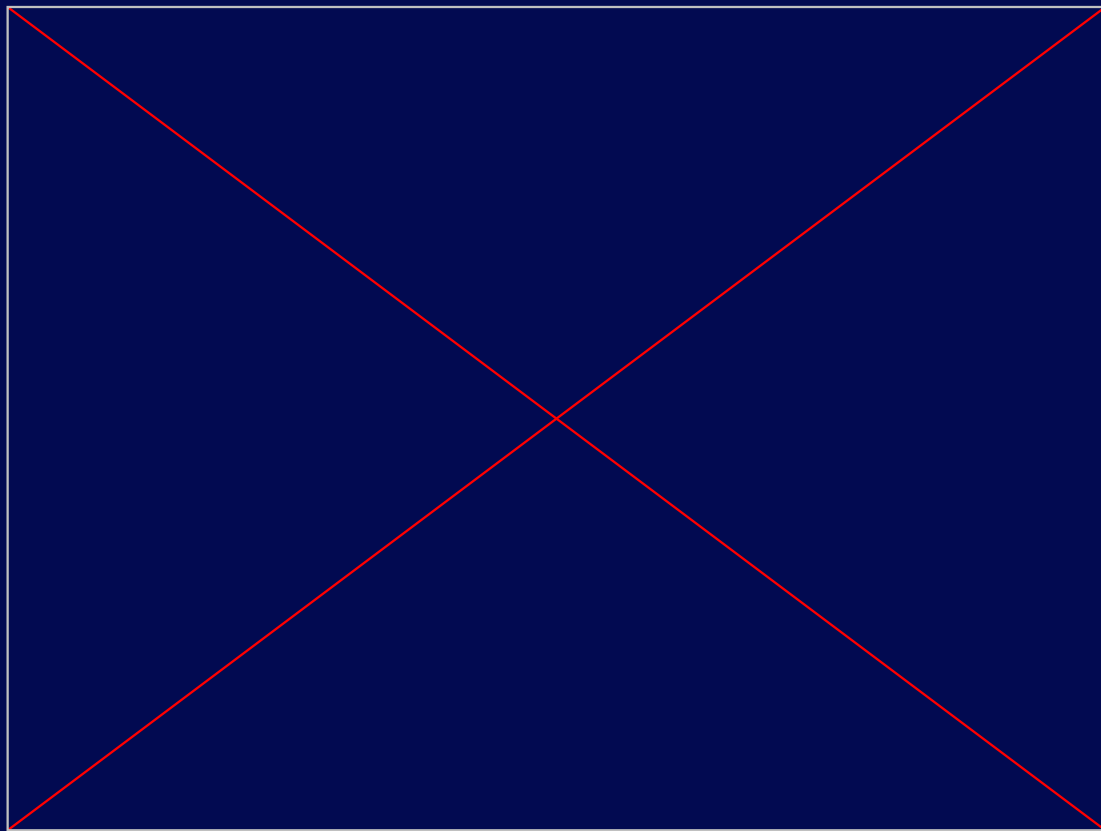
Extension Showcase



With Kiro!



Coding Locally!



Not Shown

- Easy pull-edit-push flow
- Create, edit, and run expression tests
- “Go-to” definitions
- More linting
- 18 a11y-compliant SAIL code snippets
- Massive updates to the LCP API

&

What's Next?

- Evaluate the value of local* development versus typical remote
 - If so, make local* a smoother operation (it's painful right now...)
- Extend the extension
- Better Kiro steering and integration
- In-IDE webview of Designer
- Packages & Applications
- Git usage?
- <Insert your idea here>



Spec to Slides

A Kiro Power Solution for Seamless UX to Dev Handoffs

Sonali Simhal

The Handoff Challenge



Fragmented Links

Developers often struggle with a "link jungle"—juggling Jira tickets, documentation, and various SAIL mockup versions simultaneously.



Single Source of Truth

When changes occur, identifying the verified requirement is difficult. Teams need a centralized "hub" for every interaction.

Introducing the Power

Automated Translation

Transform a markdown spec document into a comprehensive slide deck. Each feature is mapped to its own slide with dedicated space for context.

- ✓ Feature titles
- ✓ Interaction callouts
- ✓ SAIL link placeholders



Spec to Slides

Transform a markdown spec document into a PowerPoint deck with one slide per u...
by Sonali Simhal

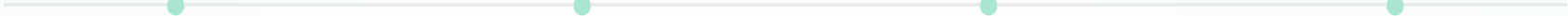
[Try power](#)[Uninstall](#)[Check for updates](#)

Description

[#spec](#)[#slides](#)[#powerpoint](#)[#pptx](#)[#user story](#)[#breakdown](#)[#presentation](#)[#product](#)

Transform a markdown spec document into a PowerPoint deck with one slide per user story. Each slide includes the story title, a SAIL placeholder link, a large screenshot placeholder, and annotation callout boxes — ready to upload to Google Slides.

The Automated Path



Share Spec

Pass your markdown file
path to Kiro.

Kiro Breakdown

Analyze story points and
acceptance criteria.

Confirm & Export

Review the outline and
generate the .pptx.

Upload & Handoff

Import to Google Slides
for the final source of
truth.



Kiro

I've read the spec. Here's the detailed slide breakdown:

Cover — CW: Implement Semantic ("Smart") Search for Document Data

[DIVIDER] Global Smart Search Entry Point |

A new Smart Search tab is added to the CW main menu, giving contracting personnel access to search across documents, record data, or both throughout all of Contract Writing.

1. **Main Menu — Smart Search Tab** — New tab visible in CW main navigation
 2. **Smart Search — Search Type Selector** — UI showing the 3 search modes: Documents, Record Data, Documents & Record Data
-

[DIVIDER] Search by Documents | When searching by Documents only, the smart search evaluates the content of all documents in CW across all records, without evaluating procurement record data.

3. **Document Search — Empty State** — Search input with Documents mode selected, no results yet
4. **Document Search — Results** — Results list showing document name and matching snippet per result, sorted by relevance score

STEP 1: PARSE

Context Capture

Kiro reads the spec and provides a structured breakdown of covers, dividers, and stories.

Total: 1 cover + 3 dividers + 19 story slides = 23 slides

Does this level of detail work? Let me know if you'd like to:

- Add or remove any slides
- Rename or reword anything
- Split or combine any screens
- Adjust acceptance criteria

Once confirmed, I'll generate the spec file and produce the `.pptx`.

STEP 2: PLAN

Structure Review

Review the total slide count and detail level before final generation.

FEATURE

Global Smart Search — Entry & Configuration

Accessing the Smart Search capability from the CW main menu and
configuring search parameters

Smart Search — Search Type: Documents Selected

SAIL

"Documents" option clearly selected

[Screenshot Placeholder]

CW Main Menu — Smart Search Navigation

SAIL

Main menu includes a dedicated way to access Smart Search

Visible to all users with existing search permissions

Consistent placement with other CW navigation elements

[Screenshot Placeholder]

STEP 3: DEPLOY

Build Success

Instant generation of a developer-ready .ptpx with
widescreen 16:9 layouts.

Detailed Handoff Plan

Section	Feature Item	Documentation Focus
Main Menu	Smart Search Nav	Entry points & visible permissions
Search Selector	UI Search Modes	Documents vs. Record Data selection
Results	Document State	Empty state vs. result list ranking

* Based on the Kiro-generated breakdown for Document Search features.

Handoff Components



Screenshots

Visual anchors for every feature, showing exactly how the SAIL components should look.



SAIL Links

Direct access to the live mockups for developers to inspect property by property.



Interaction Notes

Detailed acceptance criteria and notes explaining the logic behind the UI.

Workflow Evolution

Handoff Pillar	Manual Effort	Kiro Power
Context Capture	4-6 Hours	30 Seconds
Logic Alignment	Frequent Slack Rework	Zero-interpretation Specs
Visual Mapping	Copy-Paste Tedium	Automated Story Frames
Verification	Meeting Dependent	Asynchronous Source of Truth

Solutions Feature Doc Genie

Automated Release Documentation via Atlas MCP + Kiro Power

Meenakshi Sundaram Velmurugan



Workflow Transformation: Problem vs. Solution

The Problem

- 6 documents required per feature release
- Each takes 2–4 hours → 12–24 hours total
- Info scattered: Specs, JIRA, Sheets, codebase
- Same codebase exploration repeated per doc
- No spec validation — gaps found late
- Inconsistencies across releases

The Solution

- One prompt: spec + JIRA + breakdown + product
- Validates spec readiness — flags gaps first
- Atlas MCP explores codebase (single pass)
- Generates 6 template-compliant markdown docs
- Smart defaults from Atlas (marked for PO)
- Optional: styled Google Docs export

Architecture: How it works

Collect & Validate

Spec, JIRA, Sheets



Atlas Exploration

Single pass, reused



Generate Documents

Template-driven



Export to Drive

Pandoc → Google Docs

8 Atlas Tool Calls (Single Pass)

`get_app_overview` → app structure & stats

`get_hub_objects` → shared/high-impact components

`search_bundles` → feature bundles by keyword

`search_objects` → record types, integrations, plugins

`get_bundle` → detailed bundle contents

`get_dependencies` → object relationships & blast radius

`list_releases` → release history

`get_object_code` → SAIL source code

What This Enables

- Auto-fill data model (record types, CDTs)
- Identify integrations & plugins for security
- Detect portals for perf & security analysis
- Map cross-app dependencies & blast radius
- Infer data volumes from app maturity
- Estimate concurrent users from hub traffic
- Extract SAIL code for technical design
- Trace dependency paths for ADR consequences

6 Documents + Smart Features

Document	Req	Key Sections Auto-Filled
FIP	Yes	Breakdown, Stakeholders, Security, Risks
Technical Design	Yes	Data Model, Components, Integrations, SAIL
Performance Review	Yes	Data Load, Concurrency, Portals, Testing
Security Review	Yes	Object Matrix, Row-Level, Plugins, Integrations
Architecture Overview	Opt	C4 Diagrams, MoSCoW, API Contracts
ADR	Opt	Context, Options + LoE, Blast radius

Supported Products: ProcureSight | GAM Solutions | Case Management Studio

Tech Stack: Kiro Power + Atlas MCP (Docker) + gws CLI + JIRA MCP + Pandoc

Spec Readiness Check

- Validates before generating
- Checks personas, scope, UX, NFRs
- Gaps → [TODO] placeholders

Smart Defaults

- Data volumes from Atlas record types
- Concurrent users from hub traffic
- Portals/Plugins/Integrations: definitive
- Marked: "Confirm with PO"

Guardrails

- Anti-hallucination: facts from Atlas only
- Progress tracking: real-time status
- Graceful degradation: never aborts
- Cross-app analysis supported

Impact

TIME REDUCTION

12–24 hours → **Under 2 minutes**

Compliance & Accuracy

100% template compliance with official Appian Solutions templates.

Early Risk Mitigation

Spec gaps caught early — before implementation begins.

Data Fidelity

No hallucinated content — all facts from Atlas, gaps explicitly marked.

Strategic Analysis

Cross-app dependency analysis included (multi-app features).

REUSABLE ACROSS: ProcureSight | GAM Solutions | Case Management Studio

DEMO

One prompt → Six documents → Under 2 minutes

01 Single prompt with feature spec + JIRA +
breakdown

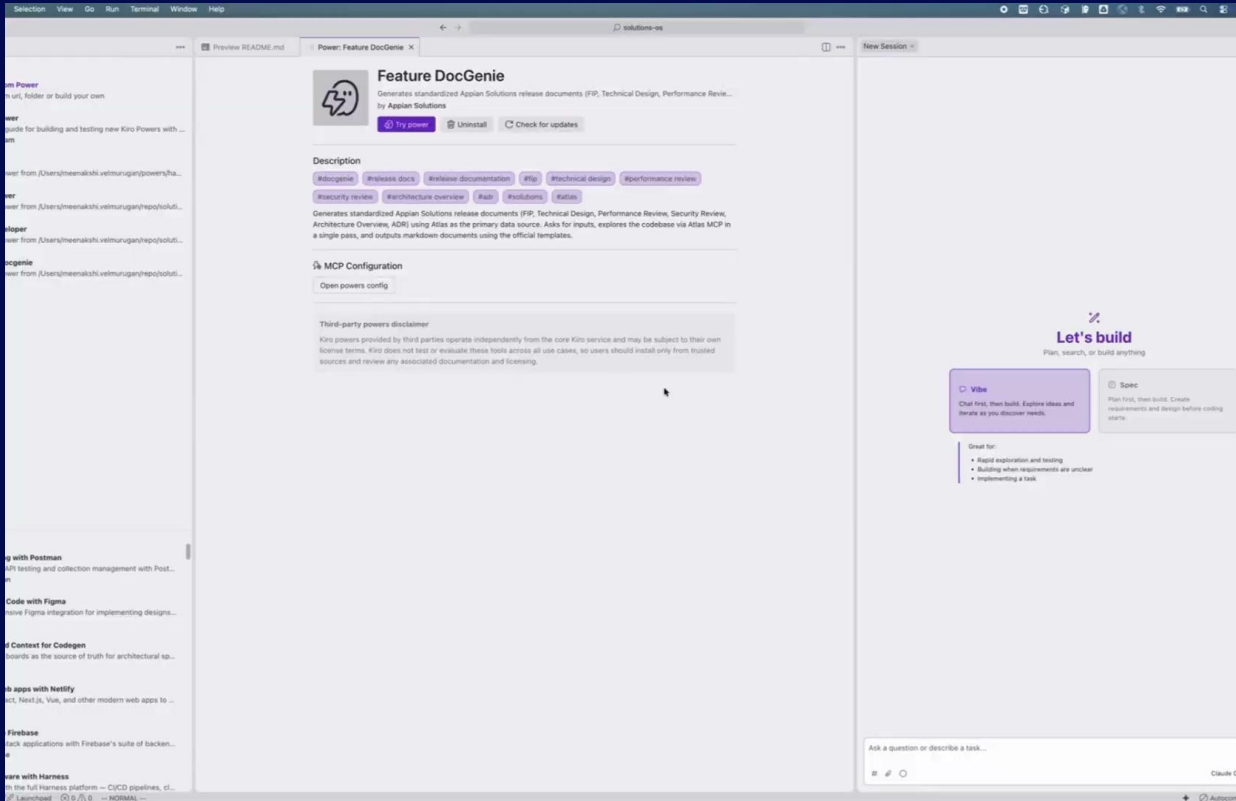
02 Spec readiness check → flags gaps

03 Atlas exploration → progress tracker updates live

04 Generated markdown output

05 Google Docs export (styled, in Drive folder)

DEMO



Sample - Documents Feedback - Sheet

Automate ERD & Release Documentation

The Project Team



Ranjith G K



Revathi Jayabalan



Gautham Muthukumar

Problem Statement: Manual ERD & Release Specific Documentation

Generating ERDs and release documentation for each product version is a **manual, repetitive process** that scales poorly.

The Challenges

- SQL export from Database and Import in Lucid
- Manually drawing ERD diagrams in Lucid
- Adjusting the Connectors, Grouping of entities
- Cross-referencing SQL scripts for changes
- Formatting release notes from scratch
- Context switching (SQL, Lucid, Docs)

Operational Pain

- Consistency gaps across 15+ products
- Repeating same workflow every release
- Scalability issues across the portfolio

Time Cost: **2-4 Hours**

per product per release, multiplied across the portfolio

Introducing the ERD Generator Power - Release Specific Details Summary

One line → three deliverables → **under 2 minutes**

INPUT PROMPT

"Hey ERD, what changed in RM?"

- No manual diagramming
- No copy-paste
- No context switching

PROFESSIONAL OUTPUTS

1. Simple ERD

PK/FK relationships only (Lucidchart)

2. Complex ERD

All columns with data types (Lucidchart)

3. Customer Release Notes

Formatted Google Doc ready for stakeholders



Stakeholder-ready documentation
generated in Google Docs format.

How It Works

1. Trigger

User says "Hey ERD, what changed in RM 2.5?"

Kiro activates the ERD Generator Power via workspace steering rules

2. Parse

Reads SQL DDL file from power's sql/ directory

Extracts CALL statements for tables, FKs, and deprecations

3. Generate Simple ERD

Builds Lucid Standard Import JSON

PK/FK columns only, Crow's foot notation, clean connectors

4. Generate Complex ERD

Same structure, all columns with data types

Lengths, NOT NULL constraints, and DEFAULT values visible

5. Create Google Doc

Runs Python script via subprocess

Inserts formatted release notes with headings and tables

6. Deliver

Final summary presentation

Presents all three links to the user in a summary table



ERD Generator Power - Create ERD with Lucid

A line → One Document with the list of simple ERDs

INPUT PROMPT

"Hey ERD, generate simple ERD for RM "

- No manual diagramming
- No copy-paste
- No context switching

PROFESSIONAL OUTPUTS

1. Doc with list of Simple ERDs

PK/FK relationships only (Lucidchart)

Ex: Execute them in Batches (155 Tables , 199 Foreign Keys)

All ERDs include a diagram key and utilize color-coded logical grouping across entities to improve readability and architectural clarity.

Create ERD with Lucid

Comprehensive automated generation of ERD documentation and simple entity relationships.

Color Legend

Blue (#DAE8FC) - Core Tables
Pink (#F8CECC) - Reference Tables
Orange (#FCE4D6) - BIC Contract Tables
Light Orange (#FFE6CC) - Procurement Tables
Green (#D5E8D4) - Questionnaire (QNM) Tables
Light Green (#E2EFDA) - Task Management (TMG) Tables
Purple (#E1D5E7) - Audit Tables
Yellow (#FFF2CC) - Messaging Tables
Pink (#FFE6F0) - SharePoint/Migration Tables
Mint (#C6EFCF) - AI Tables

15. QNM Ref & Template (Part 2) (5 tables) - Green
<https://lucid.app/lucidchart/769aad5b-0d8c-495c-bed0-bc9e8946fa76/edit>
16. QNM Audit (Part 1) (7 tables) - Green
<https://lucid.app/lucidchart/83dcfe0f-8c22-4326-804f-01f84775e08/edit>
17. QNM Audit (Part 2) (7 tables) - Green
<https://lucid.app/lucidchart/14f468cd-b428-48d0-9e0d-3212d52e576/edit>
18. QNM Audit (Part 3) (7 tables) - Green
[Pending generation]
19. QNM Audit (Part 4) (7 tables) - Green

8. SharePoint & AI (Part 1) (7 tables) - Pink/Mint
<https://lucid.app/lucidchart/00d23f46-17d3-4c5a-8ff0-d6a7f04bd833/edit>
9. SharePoint & AI (Part 2) (7 tables) - Pink/Mint
<https://lucid.app/lucidchart/ee0338ad-91f9-4c0d-804b-8d917b989544/edit>
10. Procurement (Part 1) (7 tables) - Orange
<https://lucid.app/lucidchart/59fe8313-9959-4b1e-b0c3-05be86e39abf/edit>
11. Procurement (Part 2) (7 tables) - Orange
<https://lucid.app/lucidchart/717bc09b-2822-489f-a91b-204e3c340173/edit>
12. Procurement (Part 3) (5 tables) - Orange
<https://lucid.app/lucidchart/d710ad93-6585-4c29-8415-4d6998a4bf21/edit>
13. QNM Core (7 tables) - Green
<https://lucid.app/lucidchart/4c5068f0-37ec-4af1-b9fa-2d04d74cd754/edit>
14. QNM Ref & Template (Part 1) (7 tables) - Green
<https://lucid.app/lucidchart/9816971d-6026-42aa-83a1-de56724cd39/edit>

Core & Reference (31 tables, 70 FKs)

Central requirement entity with relationships to line items, delivery, pricing, addresses, documents, funding, key dates, and codes. The REQUIREMENT table is the hub with 12+ FK relationships.

Audit (12 tables, 9 FKs)

Field-level audit trail tables for requirements, documents, line items, AAC addresses, and NIGP codes. Each audit table links to its parent audit record.

SharePoint & AI (14 tables, 15 FKs)

Output Specifications

- Automated Lucidchart import logic
- Crow's foot notation standard
- Structured relationship mapping
- Seamless integration with Google Docs notes
- Error-free DDL parsing and visualization

Creation of ERD (Full) RM 155 Tables 199 FK

What We Tried

- **lucid_add_block:** Only basic shapes; no support for complex table cells/headers.
- **lucid_create_diagram:** Always creates a NEW document; no targeting existing pages.
- **lucid_edit_item:** Can modify existing items but cannot insert new structured tables.

Impact

- **Fresh Docs Every Time:** Each generation creates a disconnected Lucid file.
- **No Incremental Updates:** Impossible to append to a team's master ERD.
- **Full Regeneration Required:** Must rebuild the entire diagram for mid-release changes.

Workaround (Current)

- **Versioned Documents:** Generate complete ERDs per release as standalone files.
- **Hybrid Edits:** Use API for minor text/connector edits; manual drag-and-drop for new tables.
- **Manual Maintenance:** Human intervention needed for "one-off" additions.

Future Strategy

- **Monitor API:** Watch for Lucid MCP updates for structured shape support.
- **Standard Import:** Explore page-level appends via import formats.
- **Rebuild Workflow:** Design a "read-rebuild-replace" loop for existing diagrams.

Tool Evaluation

Evaluated three ERD tools for MCP-based automation

Lucid (Primary — Selected)

- ✓ Already licensed (\$0 cost)
- ✓ 70+table support
- ✓ Full MCP CRUD support
- ✓ Professional output
- ✓ Team familiarity
- ✓ Enterprise security

Mermaid (Secondary — Free)

- ✓ Git-native & text-based
- ✓ Inline IDE rendering
- ✓ Free local MCP
- ✗ Poor readability @ 90 tables
- ✗ No entity grouping
- ✗ Lacks Lucid's visual polish

Miro (Not Recommended)

- ✗ Redundant with Lucid
- 2x cost (\$20 vs \$10/user)
- ✗ No additional value

Why Lucid - ERD Generator Power

Intuitive UI

Streamlines complex architectural mapping — drag, connect, and organize large data models without fighting the tool.

Magnetic Connectors

Lines auto-route, snap to edges, and re-adjust when tables move. No manual management — even with 90+ tables.

Dedicated ERD Shape Library

Purpose-built table shapes with PK/FK notation, column rows, and data type fields with Crow's foot notation.

Already Licensed & Familiar

\$0 additional cost. No onboarding, no new tool to learn, and no vendor approval process required.

THE RESULT: 2–4 hours reduced to in minutes.

Manual diagramming, SQL cross-referencing, and formatting eliminated. One prompt, professional output.

AI-assisted Kiro Knowledge Base Maintenance

Colin Hutchison

The Problem

- AI generates a lot of detailed content quickly
- All this content requires maintenance

The Solution - AI-assisted KB Maintenance

- Prerequisite: Provide the latest application packages to Kiro with each release
- Kiro steering file
 - Asks the user if they need to pull the latest from Solutions OS first
 - Systematically reviews
 - Public Docs
 - Existing KB files (steering, domain, etc.)
 - The application code to validate the changes and understand them technically
 - Updates each file
 - Quality checklist to ensure each file is updated based on a repeatable process
 - Additional cross-document review to ensure each document adheres to the document intent defined in the [structure.md](#) file and duplicate content is minimized
 - Summarization of changes
 - Final prompt asking the user if the changes should be committed and pushed back to SolutionsOS using another Kiro Agent Hook

Outcome & What's Next

- Documents updated consistently for each release with minimal user input
- N-2 and older release packages removed automatically by Kiro
- Kiro hooks to automate pulling from GitLab and committing, pushing, and creating an MR in GitLab via chat with minimal user input

What's Next

- Work with Solutions PMs to repeat this process to test
- Explore with PEs, QEs, UXDs if they have a similar use case for maintaining their Kiro context files
- Iterate on the process steering file as needed and push to Solutions OS
- Technical review of the Kiro Agent Hooks for pulling and pushing to GitLab